arXiv:2412.07187v1 [cs.LG] 10 Dec 2024

A New Federated Learning Framework Against Gradient Inversion Attacks

Pengxin Guo[1]*, Shuang Zeng[2]*, Wenhao Chen[1], Xiaodan Zhang[3], Weihong Ren[4], Yuyin Zhou[5], Liangqiong Qu[1]†

[1]School of Computing and Data Science, The University of Hong Kong
[2] Department of Mathematics, The University of Hong Kong
[3] College of Computer Science, Beijing University of Technology
[4] School of Mechanical Engineering and Automation, Harbin Institute of Technology, Shenzhen
[5] Department of Computer Science and Engineering, UC Santa Cruz
{guopx, zengsh9}@connect.hku.hk, wc365@cam.ac.uk, zhangxiaodan@bjut.edu.cn, renweihong@hit.edu.cn, zhouyuyiner@gmail.com, liangqqu@hku.hk

Abstract

Federated Learning (FL) aims to protect data privacy by enabling clients to collectively train machine learning models without sharing their raw data. However, recent studies demonstrate that information exchanged during FL is subject to Gradient Inversion Attacks (GIA) and, consequently, a variety of privacy-preserving methods have been integrated into FL to thwart such attacks, such as Secure Multi-party Computing (SMC), Homomorphic Encryption (HE), and Differential Privacy (DP). Despite their ability to protect data privacy, these approaches inherently involve substantial privacy-utility trade-offs. By revisiting the key to privacy exposure in FL under GIA, which lies in the frequent sharing of model gradients that contain private data, we take a new perspective by designing a novel privacy preserve FL framework that effectively "breaks the direct connection" between the shared parameters and the local private data to defend against GIA. Specifically, we propose a Hypernetwork Federated Learning (HyperFL) framework that utilizes hypernetworks to generate the parameters of the local model and only the hypernetwork parameters are uploaded to the server for aggregation. Theoretical analyses demonstrate the convergence rate of the proposed HyperFL, while extensive experimental results show the privacy-preserving capability and comparable performance of HyperFL. Code is available at https://github.com/Pengxin-Guo/HyperFL.

1 Introduction

Deep Neural Networks (DNN) have achieved remarkable success on a variety of computer vision tasks, relying on the availability of a large amount of training data (Krizhevsky, Sutskever, and Hinton 2012; He et al. 2016; Dosovitskiy et al. 2021; Liu et al. 2021; Wang et al. 2023). However, in many real-world applications, training data is distributed across different institutions, and data sharing between these entities is often restricted due to privacy and regulatory concerns. To alleviate these concerns, Federated Learning (FL) (McMahan et al. 2017; Li et al. 2020a; Qu et al. 2022; Zeng et al. 2024; Guo et al. 2024; Zhang et al. 2024) has emerged as a promising approach that enables collaborative and decentralized training of AI models across multiple institutions without sharing of personal data externally.

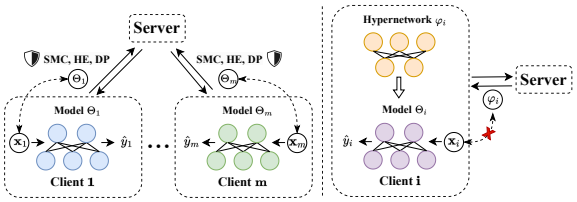


Figure 1: *Left*. Existing methods mainly explore defenses mechanisms on the shared gradients. Such mechanisms, including SMC, HE, and DP, inherently involve substantial privacy-utility trade-offs. *Right*. A novel FL framework that "breaks the direct connection" between the shared parameters and the local private data is proposed to achieve a favorable privacy-utility trade-off.

Despite the privacy-preserving capability introduced by FL, recent works (Geiping et al. 2020; Huang et al. 2021; Hatamizadeh et al. 2023) have revealed that FL models are vulnerable to Gradient Inversion Attacks (GIA) (Fredrikson, Jha, and Ristenpart 2015; Zhu, Liu, and Han 2019). GIA can reconstruct clients' private data from the shared gradients, undermining FL's privacy guarantees. To remedy this issue, various defense mechanisms have been integrated into FL, including Secure Multi-party Computing (SMC) (Yao 1982; Bonawitz et al. 2017; Mugunthan et al. 2019; Mou et al. 2021; Xu et al. 2022), Homomorphic Encryption (HE) (Gentry 2009; Zhang et al. 2020a,b; Ma et al. 2022; Park and Lim 2022) and Differential Privacy (DP) (Dwork 2006; Geyer, Klein, and Nabi 2017; McMahan et al. 2018; Yu, Bagdasaryan, and Shmatikov 2020; Bietti et al. 2022; Shen et al. 2023). These approaches primarily rely on existing defense mechanisms to enhance privacy protection against GIA without altering the FL framework, as illustrated in the left part of Figure 1. However, such mechanisms, including SMC, HE, and DP, inherently involve substantial privacy-utility trade-offs. For example, SMC and HE, while providing strong security guarantees through encrypted informa-

*These authors contributed equally.
†Corresponding author.

Copyright © 2025, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

tion exchange, entail high computation and communication costs, making them unsuitable for DNN models with numerous parameters (Bonawitz et al. 2017; Zhang et al. 2020b). Although DP approaches are easier to implement, they often fall short in providing sufficient model protection while preserving accuracy (Geyer, Klein, and Nabi 2017; McMahan et al. 2018). These trade-offs within current defense algorithms have inspired us to explore alternative privacy-preserving methods that strike a better balance between privacy and utility, leading to the central question of this paper:

Can we design a novel FL framework that offers a favorable privacy-utility trade-off against GIA without relying on existing defense mechanisms?

To this end, we revisit the key to privacy exposure in FL under GIA, which lies in the frequent sharing of model gradients that contain private data. Most efforts aim at exploring various advanced defenses mechanisms on the shared gradients to enhance privacy preservation in FL, as shown in the left part of Figure 1. In contrast, we take a new perspective striving for designing a novel privacy preserve FL framework that "breaks the direct connection" between the shared parameters and the local private data to defend against GIA. In order to achieve this, we explore the potential of hypernetworks, a class of deep neural networks that generate the weights for another network (Ha, Dai, and Le 2017), as a promising solution, as illustrated in the right part of Figure 1 and Figure 2.

Specifically, we introduce a novel Hypernetwork Federated Learning (HyperFL) framework that adopts a dual-pronged approach—network decomposition and hypernetwork sharing— to "break the direct connection" between the shared parameters and the local private data, as shown in Figure 2. In light of recent findings regarding minimal discrepancies in feature representations and considerable diversity in classifier heads among FL clients (Collins et al. 2021; Xu, Tong, and Huang 2023; Shen et al. 2023), we decompose each local model into a shared feature extractor and a private classifier, enhancing performance in heterogeneous settings and mitigating privacy leakage risks. To further strengthen privacy preservation, we employ an auxiliary hypernetwork that generates feature extractor parameters based on private client embeddings. Instead of directly sharing the feature extractor, only the hypernetwork parameters are uploaded to the server for aggregation, while classifiers and embeddings are trained locally. This auxiliary hypernetwork sharing strategy "breaks the direct connection" between shared parameters and local private data, maintaining privacy while enabling inter-client interaction and information exchange.

Remarkably, the design of HyperFL is flexible and scalable, catering to a diverse range of FL demands through its various configurations. We present two major configurations of HyperFL: (1) Main Configuration HyperFL, suitable for simple tasks with small networks, learns the entire feature extractor parameters directly (see Figure 2); (2) HyperFL for Large Pre-trained Models (denoted as HyperFL-LPM) targets for complex tasks by using pre-trained models as fixed feature extractors and generating trainable adapter parameters via a hypernetwork (Houlsby et al. 2019) (see Figure 3). Both theoretical analysis and extensive experimental results demonstrate that HyperFL effectively preserves privacy under GIA while achieving comparable results and maintaining a similar convergence rate to FedAvg (McMahan et al. 2017). We hope that the proposed HyperFL framework can encourage the research community to consider the importance of developing new enhanced privacy preservation FL frameworks, as an alternative to current research efforts on defense mechanisms front.

We summarize our contributions as follows:

- To defend against GIA, we take a new perspective by designing a novel privacy preserve FL framework that effectively "breaks the direct connection" between the shared parameters and the local private data and propose the HyperFL framework.
- We present two major configurations of HyperFL: (1) Main Configuration HyperFL, suitable for simple tasks with small networks, learns the entire feature extractor parameters directly; (2) HyperFL-LPM targets for complex tasks by using pre-trained models as fixed feature extractors and generating trainable adapter parameters via a hypernetwork.
- Both theoretical analysis and extensive experimental results demonstrate that HyperFL effectively preserves privacy under GIA while achieving comparable results and maintaining a similar convergence rate to FedAvg.

2 Related Work

Gradient Inversion Attacks. Gradient Inversion Attacks (GIA) (Fredrikson, Jha, and Ristenpart 2015; Zhu, Liu, and Han 2019) are adversarial attacks that exploit a machine learning model's gradients to infer sensitive information about the training data. It iteratively adjusts the input data based on gradients to approximate private attributes or training samples (Phong et al. 2017; Zhu, Liu, and Han 2019; Geiping et al. 2020; Yin et al. 2021; Luo et al. 2022; Geng et al. 2023; Kariyappa et al. 2023).

Privacy Protection in Federated Learning. To protect the data privacy in FL, additional defense methods have been integrated into FL, such as Secure Multi-party Computing (SMC) (Yao 1982) based methods (Bonawitz et al. 2017; Mugunthan et al. 2019; Mou et al. 2021; Xu et al. 2022), Homomorphic Encryption (HE) (Gentry 2009) based methods (Zhang et al. 2020a,b; Ma et al. 2022; Park and Lim 2022) and Differential Privacy (DP) (Dwork 2006) based methods (Geyer, Klein, and Nabi 2017; McMahan et al. 2018; Yu, Bagdasaryan, and Shmatikov 2020; Bietti et al. 2022; Shen et al. 2023). Apart from these defense methods with theoretical guarantees, there are other empirical yet effective defense strategies, such as gradient pruning/masking (Zhu, Liu, and Han 2019; Huang et al. 2021; Li et al. 2022b), noise addition (Zhu, Liu, and Han 2019; Wei et al. 2020; Huang et al. 2021; Li et al. 2022b), Soteria (Sun et al. 2020), PRECODE (Scheliga, Mäder, and Seeland 2022), and FedKL (Ren et al. 2023). However, these methods always suffer from privacy-utility trade-off problems, as illustrated in their

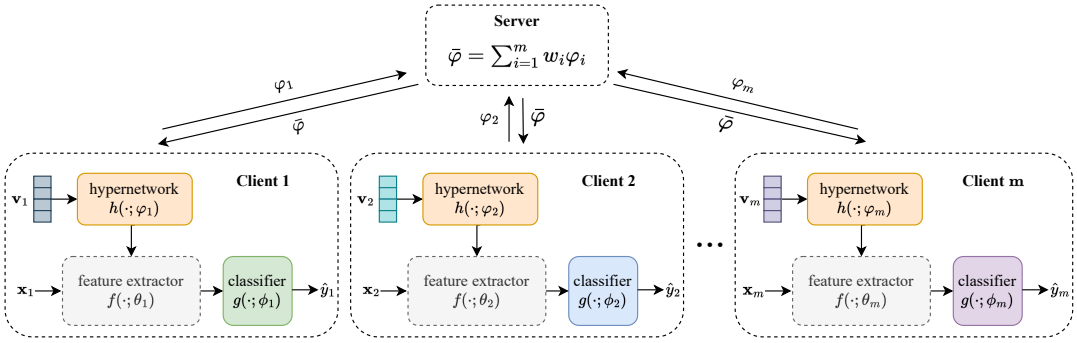


Figure 2: The proposed HyperFL framework. HyperFL decouples each client's network into the former feature extractor $f(;\theta_i)$ and the latter classifier head $g(;\phi_i)$. An auxiliary hypernetwork $h(;\varphi_i)$ is introduced to generate local clients' feature extractor $f(;\theta_i)$ using the client's private embedding vector $\mathbf{v}_i$, i.e., $\theta_i = h(\mathbf{v}_i;\varphi_i)$. These generated parameters are then used to extract features from the input x_i, which are subsequently fed into the classifier to obtain the output $\hat{y}_i$, expressed as $\hat{y}_i = g(f(x_i;\theta_i);\phi_i)$. Throughout the FL training, **only** the hypernetwork φ_i is shared, while all other components are kept private, thus effectively mitigating potential privacy leakage concerns.

papers. In contrast to these approaches, with the help of hypernetworks (Ha, Dai, and Le 2017), this work proposes a novel FL framework that effectively "breaks the direct connection" between the shared parameters and the local private data to defend against GIA while achieving a favorable privacy-utility trade-off.

Hypernetworks in Federated Learning. Hypernetworks (Ha, Dai, and Le 2017) are deep neural networks that generate the weights for another network, known as the target network, based on varying inputs to the hypernetwork. Recently, there have been some works that incorporate hypernetworks into FL for learning personalized models (Shamsian et al. 2021; Carey, Du, and Wu 2022; Li et al. 2023b; Tashakori et al. 2023; Lin et al. 2023). All of these methods adopt the similar idea that a central hypernetwork model are trained on the server to generate a set of models, one model for each client, which aims to generate personalized model for each client. Since the hypernetwork and client embeddings are trained on the server, which makes the server possessing all the information about the local models, enabling the server to recover the original inputs by GIA (see Table 3 and Figure 5 in Appendix). In contrast to existing approaches, this work presents a Hypernetwork Federated Learning (HyperFL) framework, which prioritizes data privacy preservation over personalized model generation through the utilization of hypernetworks.
A more detailed discussion on related work is provided in Appendix.

3 Method

In this section, we first formalize the FL problem, then we present our HyperFL framework.

3.1 Problem Formulation

In FL, suppose there are m clients and a central server, where all clients communicate to the server to collaboratively train their models without sharing raw private data. Each client i is equipped with its own data distribution $P_{XY}^{(i)}$ on $\mathcal{X} \times \mathcal{Y}$, where $\mathcal{X}$ is the input space and $\mathcal{Y}$ is the label space with K categories in total. Let $\ell : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}_+$ denotes the loss function given local model Θ_i and data point sampled from $P_{XY}^{(i)}$, then the underlying optimization goal of FL can be formalized as follows

$$\arg\min_{\Theta} \frac{1}{m} \sum_{i=1}^{m} \mathbb{E}_{(x,y)\sim P_{XY}^{(i)}} \left[\ell\left(\Theta_i; x, y\right)\right], \tag{1}$$

where $\Theta = \{\Theta_1, \Theta_2, \ldots, \Theta_m\}$ denotes the collection of all local models. In vanilla FL, all clients share the same parameters, i.e., $\Theta_1 = \Theta_2 = \cdots = \Theta_m$. In contrast, personalized FL allows for variation in the parameters across clients, enabling Θ_i to be different for each client.

Since the true underlying data distribution of each client is inaccessible, the common approach to achieving the objective (1) is through Empirical Risk Minimization (ERM). That is, assume each client has access to n_i i.i.d. data points sampled from $P_{XY}^{(i)}$ denoted by $\mathcal{D}_i = \left\{\left(x_i^l, y_i^l\right)\right\}_{l=1}^{n_i}$, whose corresponding empirical distribution is $\hat{P}_{XY}^{(i)}$, and we assume the empirical marginal distribution $\hat{P}_{XY}^{(i)}$ is identical to the true $P_{XY}^{(i)}$. Then the training objective is

$$\arg\min_{\Theta} \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}_i(\Theta_i), \tag{2}$$

where $\mathcal{L}_i(\Theta_i) = \frac{1}{n_i}\sum_{l=1}^{n_i} \ell(\Theta_i; x_i^l, y_i^l)$ is the local average loss over personal training data, e.g., empirical risk.

3.2 Main Configuration HyperFL

In the Main Configuration HyperFL framework, which is shown in Figure 2, each client i has a classification network parameterized by $\Theta_i = \{\theta_i, \phi_i\}$ consists of a feature extractor $f : \mathcal{X} \to \mathbb{R}^d$ parameterized by θ_i , and a classifier $g : \mathbb{R}^d \to \mathbb{R}^K$ parameterized by ϕ_i, where d is the feature dimension and K is the number of classes. Additionally, each client i has a private client embedding $\mathbf{v}_i$ and a hypernetwork h parameterized by φ_i, which is responsible for generating the parameters of the feature extractor f, i.e., $\theta_i = h(\mathbf{v}_i;\varphi_i)$. In this way, the hypernetwork can generate personalized feature extractor parameters for each

client by taking the meaningful client embedding as input. The client embeddings can be trainable vectors or fixed vectors, depending on whether suitable client representations are known in advance. In this work, we adopt trainable vectors. Then, the objective (2) can be reformulated as

$$\arg\min_{\varphi,\phi,\mathbf{v}} \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i(h(\mathbf{v}_i;\varphi_i),\phi_i), \tag{3}$$

where $\varphi = \{\varphi_1, \varphi_2, \ldots, \varphi_m\}$, $\phi = \{\phi_1, \phi_2, \ldots, \phi_m\}$, $\mathbf{v} = \{\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_m\}$. Note that the feature extractor parameters are generated by the hypernetwork and not trainable, whereas the client embedding, the hypernetwork, and the classifier parameters are trainable.

Then, in order to "breaks the direct connection" between the shared parameters and the local private data to defend against GIA while maintaining competitive performance, each client only uploads the hypernetwork parameters to the server for aggregation while keeping the classifier and the private client embedding trained locally. As illustrated in Figure 2, in each FL communication round, each client i uploads its hypernetwork parameters φ_i to the server once the local training is completed while keeps the classifier parameters ϕ_i and client embedding $\mathbf{v}_i$ local to strengthen privacy protection. Then, the server aggregate these φ_i to obtain the global $\bar{\varphi}$. Next, clients download $\bar{\varphi}$ to replace their corresponding local hypernetworks and start the next training iteration. This framework provides a natural way for sharing information across clients while maintaining the privacy of each client, by sharing the hypernetwork parameters. We will elaborate on this workflow in the following.

Local Training Procedure. For local model training at each round, we first replace the local hypernetwork parameters φ_i by the received aggregated hypernetwork parameter $\bar{\varphi}$. Then, we perform stochastic gradient decent steps to iteratively train the model parameters as follows:

- **Step 1: Fix φ_i and $\mathbf{v}_i$, update ϕ_i.** Train the classifier parameters ϕ_i by gradient descent for one epoch:

$$\phi_i \leftarrow \phi_i - \eta_g \nabla_{\phi_i}\ell\left(h(\mathbf{v_i};\varphi_i),\phi_i;\xi_i\right), \tag{4}$$

  where ξ_i denotes the mini-batch of data, η_g is the learning rate for updating the classifier parameters.
- **Step 2: Fix new ϕ_i, update φ_i and $\mathbf{v}_i$.** After getting new classifier, we proceed to update the hypernetwork parameters φ_i and client embedding $\mathbf{v}_i$ for multiple epochs:

$$\begin{aligned}\varphi_i &\leftarrow \varphi_i - \eta_h \nabla_{\varphi_i}\ell\left(h(\mathbf{v_i};\varphi_i),\phi_i;\xi_i\right)\\ \mathbf{v}_i &\leftarrow \mathbf{v}_i - \eta_v \nabla_{\mathbf{v}_i}\ell\left(h(\mathbf{v_i};\varphi_i),\phi_i;\xi_i\right),\end{aligned} \tag{5}$$

  where η_h is the learning rate for updating the hypernetwork parameters and η_v is the learning rate for updating the client embedding.

Global Aggregation. Similar to common FL algorithms, the server performs weighted averaging of the hypernetwork parameters as

$$\bar{\varphi} = \sum_{i=1}^{m} w_i\varphi_i, \tag{6}$$

where w_i is the aggregation weight for client i, usually determined by the local data size, i.e., $w_i = \frac{n_i}{\sum_{i=1}^{m} n_i}$.

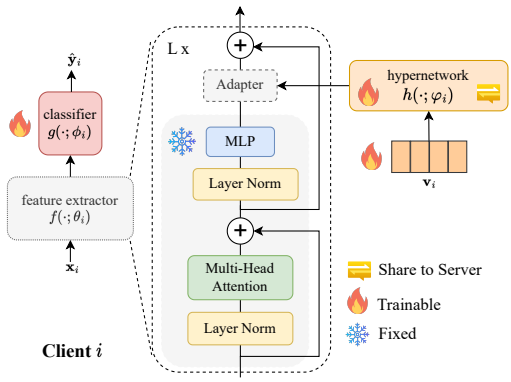


Figure 3: The proposed HyperFL-LPM framework within each client. In this framework, the weights of the pre-trained model are fixed, while only the classifier, hypernetwork, and client embedding are trainable. Note that θ here represents the parameters of the adapters.

3.3 HyperFL-LPM

When confronted with a large feature extractor, using a hypernetwork to generate the parameters can be challenging. However, in scenarios where the feature extractor is significantly sizable, there are often numerous pre-trained models that are readily available (He et al. 2016; Vaswani et al. 2017; Dosovitskiy et al. 2021; Liu et al. 2021; He et al. 2022). Consequently, we can leverage these pre-trained models and employ parameter-effective fine-tuning techniques to adapt and fine-tune them (Houlsby et al. 2019; Hu et al. 2022; Jia et al. 2022; Guo et al. 2024). To this end, we extend our HyperFL framework to address this situation and propose HyperFL for Large Pre-trained Models (HyperFL-LPM). The difference between HyperFL-LPM and Main Configuration HyperFL is the model adopted withen each client. In the HyperFL-LPM famework, instead of using the hypernetwork to generate the entire feature extractor parameters, we employ it to generate the adapter parameters to fine-tune the pre-trained models. Taking the Transformer-based pre-trained models as an example, the framework within each client is shown in Figure 3. By adopting this approach, our framework can utilize large pre-trained models as fixed feature extractors to handle complex tasks.

4 Analysis and Insights

4.1 Privacy Protection Analysis

In this section, we present a comprehensive privacy analysis of our HyperFL framework. We consider the most common and widely adopted setting, where the server is an *honest-but-curious* adversary, which obeys the training protocol but attempts to obtain the private data of clients according to model weights and updates (Liu et al. 2022; Li et al. 2023a).

Background: Gradient Inversion Attacks. Given a neural network with parameters Θ and the gradients $\nabla_\Theta\mathcal{L}_\Theta(x^*, y^*)$ computed with a private data batch (x^*, y^*), GIA tries to recover x, an approximation of x^* as:

$$\arg\min_{x} \mathcal{L}_{\text{grad}}\left(x;\Theta,\nabla_\Theta\mathcal{L}_\Theta\left(x^*,y^*\right)\right) + \alpha\mathcal{R}_{\text{aux}}\left(x\right), \tag{7}$$

where $\mathcal{L}_{\text{grad}}\left(x;\Theta,\nabla_{\Theta}\mathcal{L}_{\Theta}\left(x^{*},y^{*}\right)\right)$ is a gradient loss term used to enforce matching the gradients of recovered batch x with the provided gradients $\mathcal{L}_{\Theta}(x^{*},y^{*})$, $\mathcal{R}_{\text{aux}}\left(x\right)$ is a regularization term utilized to regularize the recovered image based on image priors, and α is a regularization coefficient. The differences between previous works lie in the choice of gradient loss terms and regularization terms (Zhu, Liu, and Han 2019; Geiping et al. 2020; Yin et al. 2021; Luo et al. 2022; Geng et al. 2023; Kariyappa et al. 2023). For example, Zhu et al. (Zhu, Liu, and Han 2019) use ℓ_2-distance as $\mathcal{L}_{\text{grad}}$ but do not use a regularization term $\mathcal{R}_{\text{aux}}$. Geiping et al. (Geiping et al. 2020) adopt cosine similarity as $\mathcal{L}_{\text{grad}}$ and the total variance as $\mathcal{R}_{\text{aux}}$. Luo et al. (Luo et al. 2022) utilize cosine similarity as $\mathcal{L}_{\text{grad}}$ and apply two types of regularization within $\mathcal{R}_{\text{aux}}$: one gradient regularization for fully connected layer and another total variation regularization for convolution features. Geng et al. (Geng et al. 2023) adopt ℓ_2-distance as $\mathcal{L}_{\text{grad}}$ and divide $\mathcal{R}_{\text{aux}}$ into three terms, total variation on the input x, clip and scale operation on the input x.

Analysis on Proposed Leakage Defense. Unlike previous FL models where the entire model parameters are uploaded to the server for aggregation, the HyperFL framework only requires each client to upload the hypernetwork parameters to the server. Therefore, the objective function of an attacker tries to recover x from the HyperFL framework should be changed as:

$$\arg\min_{x}\mathcal{L}_{\text{grad}}\left(x;\varphi,\nabla_{\varphi}\mathcal{L}_{\varphi,\phi}\left(x^{*},y^{*},\mathbf{v}^{*}\right)\right)+\alpha\mathcal{R}_{\text{aux}}\left(x\right), \quad (8)$$

where φ denotes the hypernetwork parameters, ϕ is the parameters of the classifier, $\mathbf{v}^{*}$ is the client embedding. Note that there is a major difference between objective (8) and objective (7). In objective (7), the server can obtain gradients of the entire model, while in objective (8), it can only access the gradients of the hypernetwork.

Then, given that x is only exposed to the feature extractor, obtaining the information of the feature extractor is essential to recover x. However, in HyperFL, the parameters of the feature extractor are obtained by inputting the client embedding into the hypernetwork. Therefore, it is necessary to first recover the client embedding. To achieve this pipeline, the objective (8) can be reformulated as a bi-level optimization problem:

$$\begin{aligned}&\arg\min_{x}\mathcal{L}_{\text{grad}}\left(x,\hat{\mathbf{v}};\varphi,\Delta_{\theta}\right)+\alpha\mathcal{R}_{\text{aux}}\left(x\right)\\&\text{s.t. }\hat{\mathbf{v}}=\arg\min_{\mathbf{v}}\mathcal{L}_{\text{grad}}\left(x,\mathbf{v};\varphi,\nabla_{\varphi}\mathcal{L}_{\varphi,\phi}\left(x^{*},y^{*},\mathbf{v}^{*}\right)\right),\end{aligned} \quad (9)$$

where $\mathbf{v}$ is an approximation of $\mathbf{v}^{*}$, θ is the parameters of the feature extractor that generated by the hypernertwork h, i.e., $\theta=h(\mathbf{v};\varphi)$, $\Delta_{\theta}=\theta_{t}-\theta_{t-1}$ serves as an approximation for the gradient of the feature extractor θ (Zhang et al. 2019). However, a challenge arises when solving the lower-level subproblem in objective (9). According to the chain rule, $\nabla_{\varphi}\mathcal{L}(x,y^{*},\mathbf{v})=\frac{\mathcal{L}(x,y^{*},\mathbf{v})}{\partial\phi}\frac{\partial\phi}{\partial\varphi}$. To compute the gradients of the hypernetwork, it is necessary to calculate the gradients of the classifier first [1]. However, since the classifier is trained locally and not shared with the server, it is not feasible to compute these gradients. As a result, the gradients of the hypernetwork cannot be determined, making it challenging to recover the client embedding.

One may question whether we can eliminate the need for the classifier in the process of recovering the client embedding. Drawing inspiration from the GIA procedure (Zhu, Liu, and Han 2019; Li et al. 2023a), we can replace the label information that needs to be optimized with the output of the hypernetwork in this context. By employing this approach, it's able to bypass the requirement of the classifier. Then, the lower-level subproblem in objective (9) will be reformulated as

$$\arg\min_{\mathbf{v},\theta}\mathcal{L}_{\text{grad}}\left(\theta,\mathbf{v};\varphi,\nabla_{\varphi}\mathcal{L}_{\varphi,\phi}\left(x^{*},y^{*},\mathbf{v}^{*}\right)\right), \quad (10)$$

where θ is the outputs of the hypernetwork. However, previous works have shown that simulating the optimization of both the input and output is challenging (Zhu, Liu, and Han 2019; Zhao, Mopuri, and Bilen 2020; Ma et al. 2023). Therefore, researchers propose to first identify the output and then optimize the input (Zhao, Mopuri, and Bilen 2020; Geiping et al. 2020; Zhu and Blaschko 2021; Yin et al. 2021; Ma et al. 2023; Wang, Liang, and He 2024). Specifically, they can identify the label y^{*} based on the relationship between the known gradient and label information, as y^{*} is typically low-dimensional (i.e., a simple one-hot vector) (Zhao, Mopuri, and Bilen 2020; Yin et al. 2021; Ma et al. 2023). However, the output of our hypernetwork (i.e., θ) is high-dimensional and complex. This complexity makes it challenging to identify the ground-truth output θ^{*} using the known gradient information, and consequently, recovering the embedding becomes difficult. Even if we attempt to optimize both the input and output simultaneously, solving Eq. (10) remains challenging due to the large search space (Zhu, Liu, and Han 2019; Dang et al. 2021; Huang et al. 2021; Kariyappa et al. 2023). Thus, it's challenging to recover the client embedding. Furthermore, even if the client embedding can be recovered (albeit with significant error), the input x is still difficult to recover due to the same problems (i.e., inability to infer the output first and a large search space) encountered when solving the upper-level subproblem in objective (9).

In summary, the HyperFL framework effectively safeguards data privacy, as the combination of the hypernetwork, locally trained classifier, and private client embedding renders the recovery of x using GIA unattainable, which is also demonstrated in our experiments.

4.2 Convergence Analysis

To facilitate the convergence analysis of HyperFL, we make the assumptions commonly encountered in literature (Li et al. 2020b) to characterize the smooth and non-convex optimization landscape.

Assumption 1. *$\mathcal{L}_{1},\cdots,\mathcal{L}_{m}$ are all L-smooth: for all $(\phi_{j},\varphi_{j},\mathbf{v}_{j})$ and $(\phi_{k},\varphi_{k},\mathbf{v}_{k})$, $\mathcal{L}_{i}(h(\mathbf{v}_{k},\varphi_{k}),\phi_{k})\leq\mathcal{L}_{i}(h(\mathbf{v}_{j},\varphi_{j}),\phi_{j})+((\phi_{k},\varphi_{k},\mathbf{v}_{k})-(\phi_{j},\varphi_{j},\mathbf{v}_{j}))\nabla\mathcal{L}_{i}(h(\mathbf{v}_{j},\varphi_{j}),\phi_{j})+\frac{L}{2}\|(\phi_{k},\varphi_{k},\mathbf{v}_{k})-(\phi_{j},\varphi_{j},\mathbf{v}_{j})\|_{2}^{2}$.*

Assumption 2. *Let ξ_{i}^{t} be sampled from the i-th client's local data uniformly at random at t-th training step. The variance of stochastic gradients in each client for each variable is bounded:*

[1] We assume the label information can be obtained (Geiping et al. 2020; Zhao, Mopuri, and Bilen 2020; Yin et al. 2021; Ma et al. 2023).

$\mathbb{E}\left\|\nabla_\phi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^t,\xi_i^t\right)-\nabla_\phi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^t\right)\right\|^2 \leq \sigma_i^2, \mathbb{E}\left\|\nabla_\varphi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1},\xi_i^t\right)-\nabla_\varphi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1}\right)\right\|^2 \leq \sigma_i^2$, $\mathbb{E}\left\|\nabla_{\mathbf{v}} L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1},\xi_i^t\right)-\nabla_{\mathbf{v}} L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1}\right)\right\|^2 \leq \sigma_i^2$ *for* $i=1,\cdots,m$.

Assumption 3. *The expected squared norm of stochastic gradients is uniformly bounded, i.e.,* $\mathbb{E}\left\|\nabla_\phi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^t,\xi_i^t\right)\right\|^2 \leq G^2, \mathbb{E}\left\|\nabla_\varphi L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1},\xi_i^t\right)\right\|^2 \leq G^2$, $\mathbb{E}\left\|\nabla_{\mathbf{v}} L_i\left(h\left(\mathbf{v}_i^t,\varphi_i^t\right),\phi_i^{t+1},\xi_i^t\right)\right\|^2 \leq G^2$ *for all* $i=1,\cdots,m$ *and* $t=0,\cdots,T-1$. *Here* T *denotes the total number of every client's training steps.*

Then we present the convergence rate for HyperFL.

Theorem 1. *Let Assumptions 1, 2 and 3 hold and* L, M, σ_i, G *be defined therein. Denote* $\eta_{\min}=\min\left\{\eta_g,\frac{1}{2}\eta_h,\eta_v\right\}$ *and* E *as the number of local training iterations between two communication rounds. Then we have*

$$\frac{1}{mT}\sum_{i=1}^{m}\sum_{t=1}^{T}\mathbb{E}\left[\left\|\nabla\mathcal{L}_i^t\right\|^2\right]\leq 2\sqrt{\frac{LMG^2D}{2T}}, \tag{11}$$

where $\mathcal{L}_i^0-\mathcal{L}_i^*\leq D,\forall i$, *and* $\eta_g^2+\eta_v^2+(E-1)\eta_h^2+\frac{E-1}{L}\eta_h\leq M\eta_{min}^2$.

According to Theorem 1, we can obtain an $O(\frac{1}{\sqrt{T}})$ convergence rate towards the stationary solution under smooth and non-convex conditions. This convergence rate is comparable to that of FedAvg in the non-convex scenario (Yu, Yang, and Zhu 2019). Furthermore, we can expedite the convergence for Polyak-Lojasiewicz (PL) functions (Karimi, Nutini, and Schmidt 2016), which are commonly encountered in non-convex optimization scenarios.

Assumption 4. *A function* f *is* μ*-PL function if for some* $\mu>0$, *it satisfies*

$$\|\nabla f(x)\|^2\geq 2\mu\left(f(x)-\inf_{x'}f(x')\right),\quad \forall x.$$

We assume all $\mathcal{L}_1,\cdots,\mathcal{L}_m$ *are* μ*-PL functions, and simply denote* $\inf_{x'}f(x')$ *by* f^*.

Corollary 1. *With assumptions as well as* η_{min}, L, M *and* D *defined in Theorem 1 and extra Assumption 4, we have*

$$\begin{aligned}&\mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i\left(h\left(\mathbf{v}_i^t;\varphi_i^t\right),\phi_i^t\right)\right]-\mathcal{L}^*\\ &\leq\left(1-2\eta_{\min}\mu\right)^{t+1}D+\eta_{\min}\frac{LMG^2}{4\mu}.\end{aligned} \tag{12}$$

If we set $\eta_{\min}\leq\frac{\mu\epsilon}{LMG^2}$, *after* $O\left(\frac{1}{\epsilon}\log\left(\frac{1}{\epsilon}\right)\right)$ *steps, we have that*

$$\mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i\left(h\left(\mathbf{v}_i^t;\varphi_i^t\right),\phi_i^t\right)\right]-\mathcal{L}^*\leq\epsilon. \tag{13}$$

When employing PL functions, the convergence rate of HyperFL is faster than that achieved solely through smoothness assumptions.

5 Experiments

5.1 Experimental Setup

Datasets. For the Main Configuration HyperFL, we evaluate our method on four widely-used image classification datasets: (1) EMNIST (Cohen et al. 2017); (2) Fashion-MNIST (Xiao, Rasul, and Vollgraf 2017); (3) CIFAR-10 (Krizhevsky, Hinton et al. 2009); and (4) CINIC-10 (Darlow et al. 2018). For the HyperFL-LPM, we evaluate our method on the EMNIST (Cohen et al. 2017) and CIFAR-10 (Krizhevsky, Hinton et al. 2009) datasets.

Model Architectures. For the Main Configuration HyperFL, simlar to (Xu, Tong, and Huang 2023), we adopt two different CNN target models for EMNIST/Fashion-MNIST and CIFAR-10/CINIC-10, respectively. For the HyperFL-LPM, we adopt the ViT (Dosovitskiy et al. 2021) and ResNet (He et al. 2016) pre-trained on the ImageNet dataset (Deng et al. 2009) as the feature extractor. The hypernetworks of HyperFL and HyperFL-LPM both are a fully-connected neural network with one hidden layer, multiple linear heads per target weight tensor. The client embeddings are learnable vectors with dimension equals 64.

Compared Methods. For the Main Configuration HyperFL, we compare the proposed method with the following approaches: (1) Local-only; (2) FedAvg (McMahan et al. 2017); (3) pFedHN (Shamsian et al. 2021); and some DP-based FL methods, including (4) DP-FedAvg (McMahan et al. 2018); (5) PPSGD (Bietti et al. 2022); and (6) CENTAUR (Shen et al. 2023). For the HyperFL-LPM, we compare our method with (1) Local-only with fixed feature extractor; (2) Local-only with adapter fine-tuning; (3) FedAvg with fixed feature extractor; and (4) FedAvg with adapter fine-tuning.

Training Settings. We employ the mini-batch SGD (Ruder 2016) as a local optimizer for all approaches, and the number of local training epochs is set to 5. The number of global communication rounds is set to 200 for all datasets. Average test accuracy of all local models is reported for performance evaluation.

For privacy evaluation, we adopt the widely used IG (Geiping et al. 2020), state-of-the-art ROG (Yue et al. 2023), and a tailored attack method for our defense framework to recover the input images. More details about experimental setup are provided in Appendix.

5.2 Experimental Results

Performance Evaluation. As demonstrated in Table 1, the performance of all the compared DP-based FL methods is inferior to FedAvg and Local-only. This is due to the incorporation of DP mechanisms, which adversely affect model usability and result in decreased performance. In contrast, our proposed HyperFL consistently surpasses these methods across various datasets, demonstrating its outstanding utility. Notably, HyperFL excels in both situations where Local-only outperforms (i.e., Fashion-MNIST and CINIC-10) and where FedAvg prevails (i.e., EMNIST and CIFAR-10). This further highlights HyperFL's adaptability, excelling in both centralized FL scenarios and cases requiring personalization. Furthermore, the learned client embed-

dings, which are meaningful, can be found in the Appendix. Although pFedHN outperforms our method in two scenarios, it exhibits poor defense capability against GIA, as illustrated in Table 3.

Method	EMNIST		Fashion-MNIST		CIFAR-10		CINIC-10	
	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients	20 clients	100 clients
Local-only	73.41	75.68	85.93	87.01	65.47	66.11	63.60	64.84
FedAvg	72.77	78.87	85.67	88.11	70.02	76.24	57.00	59.11
pFedHN	**80.86**	77.37	87.64	89.80	70.18	**80.07**	63.88	70.36
DP-FedAvg	35.12	45.73	59.88	68.29	29.12	32.03	27.30	29.94
CENTAUR	68.82	67.24	83.07	79.77	50.85	51.86	48.82	51.01
PPSGD	71.16	71.18	84.47	82.94	52.17	53.92	49.98	52.91
HyperFL	76.29	**80.22**	**88.28**	**90.41**	**73.03**	78.73	**66.74**	**72.21**

Table 1: The comparison of final test accuracy (%) of different methods on various datasets. We apply full participation for FL system with 20 clients, and apply client sampling with rate 0.3 for FL system with 100 clients.

The performance of HyperFL-LPM compared with Local-only and FedAvg is shown in Table 2. From this table, we can see that HyperFL-LPM can achieve comparable performance to baseline adapter fine-tuning methods with different pre-trained models, regardless of whether Local-only or FedAvg performs better. Further results for FedAvg with full parameter tuning (FPT) using ViT on EMNIST and CIFAR-10 are 78.46 and 97.78, respectively. It shows HyperFL-LPM is also comparable to FPT. This highlights the effectiveness of HyperFL-LPM.

	Arch	Local-only†	Local-only††	FedAvg†	FedAvg††	HyperFL-LPM
EMNIST	ResNet	72.83	80.35	68.99	75.21	80.32
	ViT	76.95	80.04	70.92	76.42	79.92
CIFAR-10	ResNet	68.57	73.57	62.35	75.57	75.03
	ViT	91.82	89.70	92.32	95.56	95.40

Table 2: The comparison of final test accuracy (%) of different methods on various datasets with 20 clients. † Fixed feature extractor. †† Adapter fine-tuning.

Privacy Evaluation. The reconstructed results of IG (Geiping et al. 2020) are provided in Table 3, while more results of ROG (Yue et al. 2023) and tailored attack method are provided in Table 5 and Figures 6 and 7 in Appendix. From Table 3, we can observe that the native FedAvg, pFedHN, and pFedHN-PC methods have a much higher risk of leaking data information (indicated by the higher PSNR and SSIM values and lower LPIPS value). This can also be seen in the reconstructed images, which closely resemble the original ones, as illustrated in Figure 5 in Appendix. Although introducing DP improves data privacy, there is a significant drop in model performance, as shown in Table 1. In contrast, HyperFL achieves a similar level of privacy protection while outperforming all DP-based methods and the native FedAvg in terms of model accuracy.

Method	EMNIST			CIFAR-10		
	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
FedAvg	32.64	0.8925	0.0526	16.16	0.6415	0.0536
pFedHN	31.24	0.8701	0.0807	16.02	0.6351	0.0504
pFedHN-PC	28.38	0.8713	0.0645	15.80	0.6247	0.4407
DP-FedAvg	7.74	0.2978	0.7051	7.90	0.2716	0.3204
CENTAUR	9.52	0.2136	0.6712	9.80	0.2723	0.2882
PPSGD	9.73	0.1889	0.6466	9.70	0.2788	0.2643
HyperFL	7.85	0.3010	0.7147	8.35	0.2732	0.3132

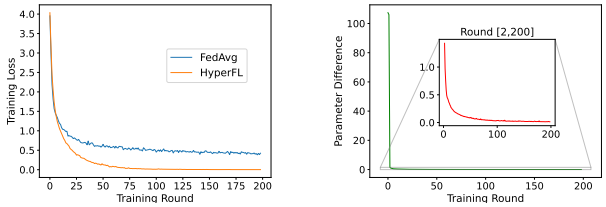
Table 3: Reconstruction results of IG.

Training Efficiency. To validate the training efficiency of the proposed HyperFL framework, we compare the training time of HyperFL with other DP-based FL methods in Table 4. This table clearly shows the efficiency of the proposed HyperFL framework. Specifically, from this table we can see that the proposed HyperFL framework runs faster than all the compared DP-based FL methods and only slightly slower than the FedAvg method. This is because DP-based FL methods often incur additional computation cost due to their privacy-preserving mechanisms, whereas HyperFL achieves faster training by leveraging the advantages of hypernetworks, all while ensuring data privacy.

	FedAvg	DP-FedAvg	PPSGD	CENTAUR	HyperFL
# Time (s)	23	194	223	210	37

Table 4: Training time of per training round on the EMNIST dataset with 20 clients of different methods.

Convergence Evaluation. To validate to convergence of the proposed HyperFL framework, we draw the training loss of FedAvg and HyperFL in Figure 4(a) and the trend of feature extractor parameters' variation in Figure 4(b). From Figure 4(a), we can observe that HyperFL almost has the same convergence rate as FedAvg, which demonstrates the convergence property of HyperFL. Moreover, after convergence, the training loss of HyperFL is lower than that of FedAvg, which reflects why HyperFL performs better than FedAvg. Furthermore, in the later stages of the training process, the variation of the feature extractor parameters approaches zero, as depicted in Figure 4(b). This further confirms the convergence property of HyperFL.



(a) (b)

Figure 4: (a) Average training loss of different methods on the EMNIST dataset with 20 clients. (b) Parameter difference of the generated feature extractor of one client between adjacent training round on the EMNIST dataset with 20 clients.

6 Conclusion

In this paper, we propose HyperFL, a novel federated learning framework that "breaks the direct connection" between the shared parameters and the local private data to defend against GIA. Specifically, this framework utilizes hypernetworks to generate the parameters of the local model and only the hypernetwork parameters are uploaded to the server for aggregation to defend against GIA while without compromising performance or incurring heavy computation overhead. We hope that the proposed HyperFL framework can encourage the research community to consider the importance of developing enhanced privacy preservation FL frameworks, as an alternative to current research efforts on defense mechanisms front.

Acknowledgments

This work was supported by National Natural Science Foundation of China (62306253, 62206075), Guangdong Natural Science Fund-General Programme (2024A1515010233), and UCSC hellman fellowship.

References

Abadi, M.; Chu, A.; Goodfellow, I.; McMahan, H. B.; Mironov, I.; Talwar, K.; and Zhang, L. 2016. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*, 308–318.

Baxter, J. 2000. A model of inductive bias learning. *Journal of artificial intelligence research*, 12: 149–198.

Bietti, A.; Wei, C.-Y.; Dudik, M.; Langford, J.; and Wu, S. 2022. Personalization improves privacy-accuracy tradeoffs in federated learning. In *International Conference on Machine Learning*, 1945–1962. PMLR.

Bonawitz, K.; Ivanov, V.; Kreuter, B.; Marcedone, A.; McMahan, H. B.; Patel, S.; Ramage, D.; Segal, A.; and Seth, K. 2017. Practical secure aggregation for privacy-preserving machine learning. In *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 1175–1191.

Carey, A. N.; Du, W.; and Wu, X. 2022. Robust Personalized Federated Learning under Demographic Fairness Heterogeneity. In *2022 IEEE International Conference on Big Data (Big Data)*, 1425–1434. IEEE.

Cohen, G.; Afshar, S.; Tapson, J.; and Van Schaik, A. 2017. EMNIST: Extending MNIST to handwritten letters. In *2017 international joint conference on neural networks (IJCNN)*, 2921–2926. IEEE.

Collins, L.; Hassani, H.; Mokhtari, A.; and Shakkottai, S. 2021. Exploiting shared representations for personalized federated learning. In *International conference on machine learning*, 2089–2099. PMLR.

Dang, T.; Thakkar, O.; Ramaswamy, S.; Mathews, R.; Chin, P.; and Beaufays, F. 2021. Revealing and protecting labels in distributed training. *Advances in Neural Information Processing Systems*, 34: 1727–1738.

Darlow, L. N.; Crowley, E. J.; Antoniou, A.; and Storkey, A. J. 2018. Cinic-10 is not imagenet or cifar-10. *arXiv preprint arXiv:1810.03505*.

Deng, J.; Dong, W.; Socher, R.; Li, L.-J.; Li, K.; and Fei-Fei, L. 2009. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, 248–255. Ieee.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.

Dwork, C. 2006. Differential privacy. In *International colloquium on automata, languages, and programming*, 1–12. Springer.

Erdoğan, E.; Küpçü, A.; and Çiçek, A. E. 2022. Unsplit: Data-oblivious model inversion, model stealing, and label inference attacks against split learning. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society*, 115–124.

Fredrikson, M.; Jha, S.; and Ristenpart, T. 2015. Model inversion attacks that exploit confidence information and basic countermeasures. In *Proceedings of the 22nd ACM SIGSAC conference on computer and communications security*, 1322–1333.

Geiping, J.; Bauermeister, H.; Dröge, H.; and Moeller, M. 2020. Inverting gradients-how easy is it to break privacy in federated learning? *Advances in Neural Information Processing Systems*, 33: 16937–16947.

Geng, J.; Mou, Y.; Li, Q.; Li, F.; Beyan, O.; Decker, S.; and Rong, C. 2023. Improved Gradient Inversion Attacks and Defenses in Federated Learning. *IEEE Transactions on Big Data*.

Gentry, C. 2009. *A fully homomorphic encryption scheme*. Stanford university.

Geyer, R. C.; Klein, T.; and Nabi, M. 2017. Differentially private federated learning: A client level perspective. *arXiv preprint arXiv:1712.07557*.

Guo, P.; Zeng, S.; Wang, Y.; Fan, H.; Wang, F.; and Qu, L. 2024. Selective Aggregation for Low-Rank Adaptation in Federated Learning. *arXiv preprint arXiv:2410.01463*.

Ha, D.; Dai, A. M.; and Le, Q. V. 2017. HyperNetworks. In *The 5th International Conference on Learning Representations*.

Hatamizadeh, A.; Yin, H.; Molchanov, P.; Myronenko, A.; Li, W.; Dogra, P.; Feng, A.; Flores, M. G.; Kautz, J.; Xu, D.; et al. 2023. Do gradient inversion attacks make federated learning unsafe? *IEEE Transactions on Medical Imaging*.

He, K.; Chen, X.; Xie, S.; Li, Y.; Dollár, P.; and Girshick, R. 2022. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 16000–16009.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.

He, Z.; Zhang, T.; and Lee, R. B. 2019. Model inversion attacks against collaborative inference. In *Proceedings of the 35th Annual Computer Security Applications Conference*, 148–162.

He, Z.; Zhang, T.; and Lee, R. B. 2020. Attacking and protecting data privacy in edge–cloud collaborative inference systems. *IEEE Internet of Things Journal*, 8(12): 9706–9716.

Hore, A.; and Ziou, D. 2010. Image quality metrics: PSNR vs. SSIM. In *2010 20th international conference on pattern recognition*, 2366–2369. IEEE.

Houlsby, N.; Giurgiu, A.; Jastrzebski, S.; Morrone, B.; De Laroussilhe, Q.; Gesmundo, A.; Attariyan, M.; and Gelly, S. 2019. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, 2790–2799. PMLR.

Hu, E. J.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; Chen, W.; et al. 2022. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*.

Huang, Y.; Gupta, S.; Song, Z.; Li, K.; and Arora, S. 2021. Evaluating gradient inversion attacks and defenses in federated learning. *Advances in Neural Information Processing Systems*, 34: 7232–7241.

Jia, M.; Tang, L.; Chen, B.-C.; Cardie, C.; Belongie, S.; Hariharan, B.; and Lim, S.-N. 2022. Visual prompt tuning. In *European Conference on Computer Vision*, 709–727. Springer.

Jiang, X.; Zhou, X.; and Grossklags, J. 2022. Comprehensive analysis of privacy leakage in vertical federated learning during prediction. *Proceedings on Privacy Enhancing Technologies*.

Karimi, H.; Nutini, J.; and Schmidt, M. 2016. Linear convergence of gradient and proximal-gradient methods under the polyak-łojasiewicz condition. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2016, Riva del Garda, Italy, September 19-23, 2016, Proceedings, Part I 16*, 795–811. Springer.

Karimireddy, S. P.; Kale, S.; Mohri, M.; Reddi, S.; Stich, S.; and Suresh, A. T. 2020. Scaffold: Stochastic controlled averaging for federated learning. In *International conference on machine learning*, 5132–5143. PMLR.

Kariyappa, S.; Guo, C.; Maeng, K.; Xiong, W.; Suh, G. E.; Qureshi, M. K.; and Lee, H.-H. S. 2023. Cocktail party attack: Breaking aggregation-based privacy in federated learning using independent component analysis. In *International Conference on Machine Learning*, 15884–15899. PMLR.

Kingma, D. P.; and Ba, J. 2015. Adam: A method for stochastic optimization. *ICLR*.

Krizhevsky, A.; Hinton, G.; et al. 2009. Learning multiple layers of features from tiny images.

Krizhevsky, A.; Sutskever, I.; and Hinton, G. E. 2012. Imagenet classification with deep convolutional neural networks. *Advances in neural information processing systems*, 25.

Li, B.; Gu, H.; Chen, R.; Li, J.; Wu, C.; Ruan, N.; Si, X.; and Fan, L. 2023a. Temporal Gradient Inversion Attacks with Robust Optimization. *arXiv preprint arXiv:2306.07883*.

Li, H.; Cai, Z.; Wang, J.; Tang, J.; Ding, W.; Lin, C.-T.; and Shi, Y. 2023b. FedTP: Federated Learning by Transformer Personalization. *IEEE Transactions on Neural Networks and Learning Systems*.

Li, T.; Sahu, A. K.; Talwalkar, A.; and Smith, V. 2020a. Federated learning: Challenges, methods, and future directions. *IEEE signal processing magazine*, 37(3): 50–60.

Li, X.; Huang, K.; Yang, W.; Wang, S.; and Zhang, Z. 2020b. On the Convergence of FedAvg on Non-IID Data. In *International Conference on Learning Representations*.

Li, Z.; Wang, L.; Chen, G.; Zhang, Z.; Shafiq, M.; and Gu, Z. 2022a. E2EGI: End-to-End Gradient Inversion in Federated Learning. *IEEE Journal of Biomedical and Health Informatics*, 27(2): 756–767.

Li, Z.; Zhang, J.; Liu, L.; and Liu, J. 2022b. Auditing privacy defenses in federated learning via generative gradient leakage. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 10132–10142.

Lin, Y.; Wang, H.; Li, W.; and Shen, J. 2023. Federated learning with hyper-network—A case study on whole slide image analysis. *Scientific Reports*, 13(1): 1724.

Liu, Z.; Guo, J.; Yang, W.; Fan, J.; Lam, K.-Y.; and Zhao, J. 2022. Privacy-preserving aggregation in federated learning: A survey. *IEEE Transactions on Big Data*.

Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; and Guo, B. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, 10012–10022.

Lowy, A.; and Razaviyayn, M. 2023. Private Federated Learning Without a Trusted Server: Optimal Algorithms for Convex Losses. In *The Eleventh International Conference on Learning Representations*.

Luo, Z.; Zhu, C.; Fang, L.; Kou, G.; Hou, R.; and Wang, X. 2022. An effective and practical gradient inversion attack. *International Journal of Intelligent Systems*, 37(11): 9373–9389.

Ma, J.; Naas, S.-A.; Sigg, S.; and Lyu, X. 2022. Privacy-preserving federated learning based on multi-key homomorphic encryption. *International Journal of Intelligent Systems*, 37(9): 5880–5901.

Ma, K.; Sun, Y.; Cui, J.; Li, D.; Guan, Z.; and Liu, J. 2023. Instance-wise Batch Label Restoration via Gradients in Federated Learning. In *The Eleventh International Conference on Learning Representations*.

McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; and y Arcas, B. A. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, 1273–1282. PMLR.

McMahan, H. B.; Ramage, D.; Talwar, K.; and Zhang, L. 2018. Learning Differentially Private Recurrent Language Models. In *International Conference on Learning Representations*.

Mou, W.; Fu, C.; Lei, Y.; and Hu, C. 2021. A verifiable federated learning scheme based on secure multi-party computation. In *International Conference on Wireless Algorithms, Systems, and Applications*, 198–209. Springer.

Mugunthan, V.; Polychroniadou, A.; Byrd, D.; and Balch, T. H. 2019. Smpai: Secure multi-party computation for federated learning. In *Proceedings of the NeurIPS 2019 Workshop on Robust AI in Financial Services*, 1–9. MIT Press Cambridge, MA, USA.

Nair, V.; and Hinton, G. E. 2010. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th international conference on machine learning (ICML-10)*, 807–814.

Park, J.; and Lim, H. 2022. Privacy-preserving federated learning using homomorphic encryption. *Applied Sciences*, 12(2): 734.

Phong, L. T.; Aono, Y.; Hayashi, T.; Wang, L.; and Moriai, S. 2017. Privacy-preserving deep learning: Revisited and enhanced. In *Applications and Techniques in Information Security: 8th International Conference, ATIS 2017, Auckland, New Zealand, July 6–7, 2017, Proceedings*, 100–110. Springer.

Qu, L.; Zhou, Y.; Liang, P. P.; Xia, Y.; Wang, F.; Adeli, E.; Fei-Fei, L.; and Rubin, D. 2022. Rethinking architecture design for tackling data heterogeneity in federated learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 10061–10071.

Ren, H.; Deng, J.; Xie, X.; Ma, X.; and Ma, J. 2023. Gradient leakage defense with key-lock module for federated learning. *arXiv preprint arXiv:2305.04095*.

Ruder, S. 2016. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*.

Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; Bernstein, M.; et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115: 211–252.

Scheliga, D.; Mäder, P.; and Seeland, M. 2022. Precode-a generic model extension to prevent deep gradient leakage. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 1849–1858.

Shamsian, A.; Navon, A.; Fetaya, E.; and Chechik, G. 2021. Personalized federated learning using hypernetworks. In *International Conference on Machine Learning*, 9489–9502. PMLR.

Shen, Z.; Ye, J.; Kang, A.; Hassani, H.; and Shokri, R. 2023. Share your representation only: Guaranteed improvement of the privacy-utility tradeoff in federated learning. *The Eleventh International Conference on Learning Representations*.

Sun, J.; Li, A.; Wang, B.; Yang, H.; Li, H.; and Chen, Y. 2020. Provable defense against privacy leakage in federated learning from representation perspective. *arXiv preprint arXiv:2012.06043*.

Tashakori, A.; Zhang, W.; Wang, Z. J.; and Servati, P. 2023. SemiPFL: personalized semi-supervised federated learning framework for edge intelligence. *IEEE Internet of Things Journal*.

Van der Maaten, L.; and Hinton, G. 2008. Visualizing data using t-SNE. *Journal of machine learning research*, 9(11).

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; and Polosukhin, I. 2017. Attention is all you need. *Advances in neural information processing systems*, 30.

Wang, Y.; Liang, J.; and He, R. 2024. Towards Eliminating Hard Label Constraints in Gradient Inversion Attacks. In *The Twelfth International Conference on Learning Representations*.

Wang, Y.; Wang, X.; Dinh, A.-D.; Du, B.; and Xu, C. 2023. Learning to Schedule in Diffusion Probabilistic Models. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

Wang, Z.; Bovik, A. C.; Sheikh, H. R.; and Simoncelli, E. P. 2004. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4): 600–612.

Wei, W.; Liu, L.; Loper, M.; Chow, K.-H.; Gursoy, M. E.; Truex, S.; and Wu, Y. 2020. A framework for evaluating gradient leakage attacks in federated learning. *arXiv preprint arXiv:2004.10397*.

Xiao, H.; Rasul, K.; and Vollgraf, R. 2017. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*.

Xu, B.; Wang, N.; Chen, T.; and Li, M. 2015. Empirical evaluation of rectified activations in convolutional network. *arXiv preprint arXiv:1505.00853*.

Xu, J.; Tong, X.; and Huang, S.-L. 2023. Personalized Federated Learning with Feature Alignment and Classifier Collaboration. In *The Eleventh International Conference on Learning Representations*.

Xu, Y.; Peng, C.; Tan, W.; Tian, Y.; Ma, M.; and Niu, K. 2022. Non-interactive verifiable privacy-preserving federated learning. *Future Generation Computer Systems*, 128: 365–380.

Yao, A. C. 1982. Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*, 160–164. IEEE.

Yin, H.; Mallya, A.; Vahdat, A.; Alvarez, J. M.; Kautz, J.; and Molchanov, P. 2021. See through gradients: Image batch recovery via gradinversion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 16337–16346.

Yu, H.; Yang, S.; and Zhu, S. 2019. Parallel restarted SGD with faster convergence and less communication: Demystifying why model averaging works for deep learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, 5693–5700.

Yu, T.; Bagdasaryan, E.; and Shmatikov, V. 2020. Salvaging federated learning by local adaptation. *arXiv preprint arXiv:2002.04758*.

Yue, K.; Jin, R.; Wong, C.-W.; Baron, D.; and Dai, H. 2023. Gradient obfuscation gives a false sense of security in federated learning. In *32nd USENIX Security Symposium (USENIX Security 23)*, 6381–6398.

Zeng, S.; Guo, P.; Wang, S.; Wang, J.; Zhou, Y.; and Qu, L. 2024. Tackling data heterogeneity in federated learning via loss decomposition. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, 707–717. Springer.

Zhang, C.; Li, S.; Xia, J.; Wang, W.; Yan, F.; and Liu, Y. 2020a. {BatchCrypt}: Efficient homomorphic encryption for {Cross-Silo} federated learning. In *2020 USENIX annual technical conference (USENIX ATC 20)*, 493–506.

Zhang, J.; Zeng, S.; Zhang, M.; Wang, R.; Wang, F.; Zhou, Y.; Liang, P. P.; and Qu, L. 2024. FLHetBench: Benchmarking Device and State Heterogeneity in Federated Learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 12098–12108.

Zhang, M.; Lucas, J.; Ba, J.; and Hinton, G. E. 2019. Lookahead optimizer: k steps forward, 1 step back. *Advances in neural information processing systems*, 32.

Zhang, M.; Sapra, K.; Fidler, S.; Yeung, S.; and Alvarez, J. M. 2021. Personalized Federated Learning with First Order Model Optimization. In *International Conference on Learning Representations*.

Zhang, R.; Isola, P.; Efros, A. A.; Shechtman, E.; and Wang, O. 2018. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 586–595.

Zhang, X.; Fu, A.; Wang, H.; Zhou, C.; and Chen, Z. 2020b. A privacy-preserving and verifiable federated learning scheme. In *ICC 2020-2020 IEEE International Conference on Communications (ICC)*, 1–6. IEEE.

Zhao, B.; Mopuri, K. R.; and Bilen, H. 2020. idlg: Improved deep leakage from gradients. *arXiv preprint arXiv:2001.02610*.

Zhu, J.; and Blaschko, M. B. 2021. R-GAP: Recursive Gradient Attack on Privacy. In *International Conference on Learning Representations*.

Zhu, L.; Liu, Z.; and Han, S. 2019. Deep leakage from gradients. *Advances in neural information processing systems*, 32.

A Proofs of Theoretical Results

A.1 Proof of Theorem 1

Proof. Let $\phi_i^t, \varphi_i^t, \mathbf{v}_i^t$ be the model parameters maintained in the i-th client at the t-th step. Let $\mathcal{I}_E$ be the set of global synchronization steps, i.e., $\mathcal{I}_E = \{nE \mid n = 1, 2, \cdots\}$. If $t+1 \in \mathcal{I}_E$, which represents the time step for communication, then the one-step update of HyperFL can be described as follows:

$$\begin{pmatrix} \phi_i^t \\ \varphi_i^t \\ \mathbf{v}_i^t \end{pmatrix} \xrightarrow[\text{SGD of } \phi_i^t]{} \begin{pmatrix} \phi_i^{t+1} \\ \varphi_i^t \\ \mathbf{v}_i^t \end{pmatrix} \xrightarrow[\text{SGD of } \varphi_i^t, \mathbf{v}_i^t]{} \begin{pmatrix} \phi_i^{t+1} \\ \varphi_i^{t+1} \\ \mathbf{v}_i^{t+1} \end{pmatrix}$$

$$\xrightarrow[\text{if } t+1 \in \mathcal{I}_E]{} \begin{pmatrix} \phi_i^{t+1} \\ \sum_{j=1}^m w_j \varphi_j^{t+1} \\ \mathbf{v}_i^{t+1} \end{pmatrix}.$$

For convenience, we denote the parameters in each sub-step above as follows:

$$x_i^t = \begin{pmatrix} \phi_i^t \\ \varphi_i^t \\ \mathbf{v}_i^t \end{pmatrix}, y_i^t = \begin{pmatrix} \phi_i^{t+1} \\ \varphi_i^t \\ \mathbf{v}_i^t \end{pmatrix},$$

$$x_i^{t+1,1} = \begin{pmatrix} \phi_i^{t+1} \\ \varphi_i^{t+1} \\ \mathbf{v}_i^{t+1} \end{pmatrix}, x_i^{t+1,2} = \begin{pmatrix} \phi_i^{t+1} \\ \sum_{j=1}^m w_j \varphi_j^{t+1} \\ \mathbf{v}_i^{t+1} \end{pmatrix},$$

$$x_i^{t+1} = \begin{cases} x_i^{t+1,1} & \text{if } t+1 \notin \mathcal{I}_E, \\ x_i^{t+1,2} & \text{if } t+1 \in \mathcal{I}_E. \end{cases}$$

Here, the variable $x_i^{t+1,1}$ represents the immediate result of one sub-step SGD update from the parameter of the previous sub-step y_i^t, and $x_i^{t+1,2}$ represents the parameter obtained after communication steps (if possible).
Furthermore, we denote the learning rate and stochastic gradient of step t as follows:

$$\eta = \begin{pmatrix} \eta_g \\ \eta_h \\ \eta_v \end{pmatrix},$$

$$g_i^t = \begin{pmatrix} g_{i,\phi}^t \\ g_{i,\varphi}^t \\ g_{i,\mathbf{v}}^t \end{pmatrix} = \begin{pmatrix} \nabla_\phi \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^t, \xi_i^t\right) \\ \nabla_\varphi \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^{t+1}, \xi_i^t\right) \\ \nabla_\mathbf{v} \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^{t+1}, \xi_i^t\right) \end{pmatrix},$$

$$\bar{g}_i^t = \begin{pmatrix} \bar{g}_{i,\phi}^t \\ \bar{g}_{i,\varphi}^t \\ \bar{g}_{i,\mathbf{v}}^t \end{pmatrix} = \begin{pmatrix} \nabla_\phi \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^t\right) \\ \nabla_\varphi \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^{t+1}\right) \\ \nabla_\mathbf{v} \mathcal{L}_i \left(h\left(\mathbf{v}_i^t, \varphi_i^t\right), \phi_i^{t+1}\right) \end{pmatrix},$$

where ξ_i^t is the data uniformly chosen from the local data set of client i at step t, then $\mathbb{E}\left[g_i^t\right] = \bar{g}_i^t$.

Next, we apply the inequality of the smoothness Assumption 1 to each sub-step of the one-step update for client i. Firstly, by the smoothness of $\mathcal{L}_i$, we have

$$\mathcal{L}_i\left(y_i^t\right) \leq \mathcal{L}_i\left(x_i^t\right) + \left\langle y_i^t - x_i^t, \bar{g}_{i,\phi}^t \right\rangle + \frac{L}{2}\left\|y_i^t - x_i^t\right\|^2. \quad (14)$$

For the second term on the right side of inequality (14), according to the law of total expectation, we have

$$\begin{aligned} \mathbb{E}\left[\left\langle y_i^t - x_i^t, \bar{g}_{i,\phi}^t \right\rangle\right] &= \mathbb{E}\left[\left\langle -\eta_g g_{i,\phi}^t, \bar{g}_{i,\phi}^t \right\rangle\right] \\ &= \mathbb{E}\left\{\mathbb{E}\left[\left\langle -\eta_g g_{i,\phi}^t, \bar{g}_{i,\phi}^t \right\rangle\right] | \xi_i^t\right\} \\ &= \mathbb{E}\left\{\mathbb{E}\left[\left\langle -\eta_g g_{i,\phi}^t | \xi_i^t, \bar{g}_{i,\phi}^t \right\rangle\right]\right\} \\ &= \mathbb{E}\left[\left\langle -\eta_g \bar{g}_{i,\phi}^t, \bar{g}_{i,\phi}^t \right\rangle\right] \\ &= -\eta_g \mathbb{E}\left[(\bar{g}_{i,\phi}^t)^2\right]. \end{aligned}$$

For the third term on the right side of the inequality (14), we have

$$\begin{aligned} \mathbb{E}\left[\frac{L}{2}\left\|y_i^t - x_i^t\right\|^2\right] &= \mathbb{E}\left[\frac{L}{2}\left\|-\eta_g g_{i,\phi}^t\right\|^2\right] \\ &= \eta_g^2 \frac{L}{2} \mathbb{E}\left[\left|g_{i,\phi}^t\right\|^2\right] \\ &\leq \eta_g^2 \frac{LG^2}{2}, \end{aligned}$$

where in the last inequality, we use the bounded gradient Assumption 3.
From the above inequalities, and taking the expectation of inequality (14), we can get

$$\mathbb{E}\left[\mathcal{L}_i\left(y_i^t\right) - \mathcal{L}_i\left(x_i^t\right)\right] \leq -\eta_g \mathbb{E}\left[(\bar{g}_{i,\phi}^t)^2\right] + \eta_g^2 \frac{LG^2}{2}. \quad (15)$$

Secondly, by the smoothness of $\mathcal{L}_i$, we have

$$\begin{aligned} \mathcal{L}_i\left(x_i^{t+1,1}\right) \leq & \mathcal{L}_i\left(y_i^t\right) + \left\langle x_i^{t+1,1} - y_i^t, \bar{g}_{i,\phi}^t \right\rangle \\ & + \frac{L}{2}\left\|x_i^{t+1,1} - y_i^t\right\|^2. \end{aligned} \quad (16)$$

Similar to inequality (15), taking the expectation of inequality (16), we get

$$\begin{aligned} & \mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1,1}\right) - \mathcal{L}_i\left(y_i^t\right)\right] \\ \leq & -\eta_h \mathbb{E}\left[(\bar{g}_{i,\varphi}^t)^2\right] - \eta_v \mathbb{E}\left[(\bar{g}_{i,\mathbf{v}}^t)^2\right] + (\eta_h^2 + \eta_v^2)\frac{LG^2}{2}. \end{aligned} \quad (17)$$

Thirdly, by the smoothness of $\mathcal{L}_i$, we have

$$\begin{aligned} \mathcal{L}_i\left(x_i^{t+1,2}\right) \leq & \mathcal{L}_i\left(x_i^{t+1,1}\right) + \left\langle x_i^{t+1,2} - x_i^{t+1,1}, \bar{g}_{i,\varphi}^t \right\rangle \\ & + \frac{L}{2}\left\|x_i^{t+1,2} - x_i^{t+1,1}\right\|^2. \end{aligned} \quad (18)$$

From the iterative formula of SGD, it is clear that

$$\varphi_j^{t+1} = \varphi_j^{t-E+1} - \eta_h \sum_{t_0=t-E+1}^{t} g_{j,\varphi}^{t_0}, \quad \forall j, t+1 \in \mathcal{I}_E. \quad (19)$$

Then, for the third term on the right side of inequality (18), we apply the equality (19) and take the expectation, which

yields

$$
\begin{aligned}
&\mathbb{E}\left[\frac{L}{2}\left\|x_i^{t+1,2}-x_i^{t+1,1}\right\|^2\right]\\
=&\frac{L}{2}\mathbb{E}\left[\left\|-\eta_h\sum_{j=1}^{m}w_j\sum_{t_0=t-E+1}^{t}(g_{j,\varphi}^{t_0}-g_{i,\varphi}^{t_0})\right\|^2\right]\\
\leq&\eta_h^2\frac{L}{2}\sum_{j=1}^{m}w_j\mathbb{E}\left[\left\|\sum_{t_0=t-E+1}^{t}(g_{j,\varphi}^{t_0}-g_{i,\varphi}^{t_0})\right\|^2\right]\\
\leq&\eta_h^2\frac{L}{2}\sum_{j=1}^{m}w_j\sum_{t_0=t-E+1}^{t}\mathbb{E}\left[\left\|(g_{j,\varphi}^{t_0}-g_{i,\varphi}^{t_0})\right\|^2\right]\\
\leq&\eta_h^2\frac{L}{2}\sum_{j=1}^{m}w_j\sum_{t_0=t-E+1}^{t}\mathbb{E}\left[\frac{1}{2}\left\|g_{j,\varphi}^{t_0}\right\|^2+\frac{1}{2}\left\|g_{i,\varphi}^{t_0}\right\|^2\right]\\
\leq&\eta_h^2\frac{(E-1)LG^2}{2},
\end{aligned}
$$

where in the last inequality, we use the bounded gradient Assumption 3.

For the second term on the right side of inequality (18), we take the expectation and get

$$
\begin{aligned}
&\mathbb{E}\left[\left\langle x_i^{t+1,2}-x_i^{t+1,1},\bar{g}_{i,\varphi}^t\right\rangle\right]\\
\leq&\frac{1}{2\eta_h}\left(x_i^{t+1,2}-x_i^{t+1,1}\right)^2+\frac{1}{2}\eta_h\left(\bar{g}_{i,\varphi}^t\right)^2\\
\leq&\frac{1}{2\eta_h}\eta_h^2(E-1)G^2+\frac{1}{2}\eta_h\left(\bar{g}_{i,\varphi}^t\right)^2\\
=&\eta_h\frac{(E-1)G^2}{2}+\frac{1}{2}\eta_h\left(\bar{g}_{i,\varphi}^t\right)^2,
\end{aligned}
$$

where we use the Cauchy-Schwarz inequality and the AM-GM inequality in the first inequality, and the bounded gradient Assumption 3 in the second inequality above.

Then, based on the above inequalities and taking the expectation of inequality (18), we have

$$
\begin{aligned}
&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1,2}\right)-\mathcal{L}_i\left(x_i^{t+1,1}\right)\right]\\
\leq&\eta_h^2\frac{(E-1)LG^2}{2}+\eta_h\frac{(E-1)G^2}{2}+\frac{1}{2}\eta_h\left(\bar{g}_{i,\varphi}^t\right)^2.
\end{aligned}
\tag{20}
$$

Summing up inequalities (15) and (17), we get

$$
\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1,1}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\leq-\eta^\top\mathbb{E}\left[(g_i^t)^2\right]+\|\eta\|^2\frac{LG^2}{2}.
\tag{21}
$$

Summing up inequalities (15), (17), and (20), we get

$$
\begin{aligned}
&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1,2}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\\
\leq&-\begin{pmatrix}\eta_g\\ \frac{1}{2}\eta_h\\ \eta_v\end{pmatrix}^\top\mathbb{E}\left[\left(\bar{g}_i^t\right)^2\right]\\
&+\left[\eta_g^2+\eta_v^2+(E-1)\eta_h^2+\frac{E-1}{L}\eta_h\right]\frac{LG^2}{2}.
\end{aligned}
\tag{22}
$$

Then, let $\eta_{min}=\min\{\eta_g,\frac{1}{2}\eta_h,\eta_v\}$, we have

$$
\begin{aligned}
&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1,2}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\\
\leq&-\eta_{min}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\\
&+\left[\eta_g^2+\eta_v^2+(E-1)\eta_h^2+\frac{E-1}{L}\eta_h\right]\frac{LG^2}{2}.
\end{aligned}
\tag{23}
$$

By rewriting the above inequality (23), we get

$$
\begin{aligned}
\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq&\frac{\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^{t+1}\right)\right]}{\eta_{min}}\\
&+\frac{\eta_g^2+\eta_v^2+(E-1)\eta_h^2+\frac{E-1}{L}\eta_h}{\eta_{min}}\frac{LG^2}{2}.
\end{aligned}
\tag{24}
$$

Let M be a constant that satisfies the inequality $\eta_g^2+\eta_v^2+(E-1)\eta_h^2+\frac{E-1}{L}\eta_h\leq M\eta_{min}^2$, the aforementioned inequality (24) can be further simplified as

$$
\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq\frac{\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^{t+1}\right)\right]}{\eta_{min}}+\eta_{min}\frac{LMG^2}{2}.
\tag{25}
$$

Now, by repeatedly applying inequality (25) for different values of t and summing up the results, we get

$$
\begin{aligned}
\sum_{t=1}^{T}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq&\frac{\mathbb{E}\left[\mathcal{L}_i\left(x_i^1\right)-\mathcal{L}_i\left(x_i^*\right)\right]}{\eta_{min}}\\
&+\eta_{min}\frac{LMG^2}{2}T.
\end{aligned}
\tag{26}
$$

Dividing both side of inequality (26) by T, we get

$$
\begin{aligned}
\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq&\frac{\mathbb{E}\left[\mathcal{L}_i\left(x_i^1\right)-\mathcal{L}_i\left(x_i^*\right)\right]}{\eta_{min}T}\\
&+\eta_{min}\frac{LMG^2}{2}.
\end{aligned}
\tag{27}
$$

Let us assume that $\mathcal{L}_i\left(x_i^1\right)-\mathcal{L}_i\left(x_i^*\right)\leq D,\forall i$, and we set $\eta_{min}=\sqrt{\frac{2D}{LMG^2T}}$. Then, we have

$$
\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq2\sqrt{\frac{LMG^2D}{2T}}.
\tag{28}
$$

Thus, we can get

$$
\frac{1}{mT}\sum_{i=1}^{m}\sum_{t=1}^{T}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]\leq2\sqrt{\frac{LMG^2D}{2T}}.
\tag{29}
$$

□

A.2 Proof of Corollary 1

Proof. By rewriting inequality (25), we have

$$
\begin{aligned}
&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\\
\leq&-\eta_{min}\mathbb{E}\left[\left\|\bar{g}_i^t\right\|^2\right]+\eta_{min}^2\frac{LMG^2}{2}.
\end{aligned}
\tag{30}
$$

By the PL Assumption 4, we have

$$\begin{aligned}&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\\ \leq&-2\eta_{min}\mu\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^*\right)\right]+\eta_{\min}^2\frac{LMG^2}{2}.\end{aligned}\tag{31}$$

Then,

$$\begin{aligned}&\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1}\right)-\mathcal{L}_i\left(x_i^*\right)\right]\\ =&\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^*\right)\right]+\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t+1}\right)-\mathcal{L}_i\left(x_i^t\right)\right]\\ \leq&\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^*\right)\right]-2\eta_{min}\mu\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^*\right)\right]\\ &+\eta_{min}^2\frac{LMG^2}{2}\\ =&(1-2\eta_{min}\mu)\mathbb{E}\left[\mathcal{L}_i\left(x_i^t\right)-\mathcal{L}_i\left(x_i^*\right)\right]+\eta_{min}^2\frac{LMG^2}{2}\\ \leq&\left(1-2\eta_{min}\mu\right)^2\mathbb{E}\left[\mathcal{L}_i\left(x_i^{t-1}\right)-\mathcal{L}_i\left(x_i^*\right)\right]\\ &+\sum_{\tau=0}^{1}\left(1-2\eta_{\min}\mu\right)^\tau\eta_{\min}^2\frac{LMG^2}{2}\\ \cdots\\ \leq&\left(1-2\eta_{min}\mu\right)^{t+1}\mathbb{E}\left[\mathcal{L}_i\left(x_i^0\right)-\mathcal{L}_i\left(x_i^*\right)\right]\\ &+\sum_{\tau=0}^{t}\left(1-2\eta_{min}\mu\right)^\tau\eta_{min}^2\frac{LMG^2}{2}\\ \leq&(1-\eta_{min}\mu)^{t+1}D+\eta_{min}\frac{LMG^2}{4\mu}.\end{aligned}$$

Therefore, we have

$$\begin{aligned}&\mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i\left(x_i^{t+1}\right)\right]-\mathcal{L}^*\\ \leq&(1-\eta_{min}\mu)^{t+1}D+\eta_{min}\frac{LMG^2}{4\mu}.\end{aligned}\tag{32}$$

If we set $\eta_{min}\leq\frac{\mu\epsilon}{LMG^2}$, after $O(\frac{1}{\epsilon}\log(\frac{1}{\epsilon}))$ steps, we have

$$\mathbb{E}\left[\frac{1}{m}\sum_{i=1}^{m}\mathcal{L}_i\left(x_i^{t+1}\right)\right]-\mathcal{L}^*\leq\epsilon.\tag{33}$$

□

B Generalization Bound

We further provide the generalization bound for HyperFL by employing the methodology outlined in (Baxter 2000). First, we make the following assumption:

Assumption 5. *We assume the weights of hypernetworks φ_i, the client embeddings $\mathbf{v}_i$ and the weights of classifiers ϕ_i are bounded in a ball of radius R, in which the following Lipschitz conditions hold:*

$$\begin{aligned}&|\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i^*)|\leq L_\phi||\phi_i-\phi_i^*||,\\ &|\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i)-\mathcal{L}_i(h^*(\varphi_i;\mathbf{v}_i),\phi_i)|\leq L_h||h_i-h_i^*||,\\ &||h(\varphi_i^*;\mathbf{v}_i)-h(\varphi_i;\mathbf{v}_i))||\leq L_\varphi||\varphi_i-\varphi_i^*||,\\ &||h(\varphi_i;\mathbf{v}_i^*)-h(\varphi_i;\mathbf{v}_i))||\leq L_\mathbf{v}||\mathbf{v}_i-\mathbf{v}_i^*||.\end{aligned}$$

Theorem 2. *Suppose we select m clients at each communication round. Let the hypernetwork parameter space be of dimension H, the embedding space be of dimension d and the classifier parameter space be of dimension K. Let the ϕ, $\mathbf{v}$, φ be the parameters learned from the individual dataset of clients. When Assumption 5 holds, there exists*

$$\begin{aligned}S=&\mathcal{O}\Big(\frac{d+H+K}{\epsilon^2}\log\Big(\frac{R(L_hL_\varphi+L_hL_v+L_\phi)}{\epsilon}\Big)\\ &+\frac{1}{m\epsilon^2}\log\frac{1}{\delta}\Big),\end{aligned}\tag{34}$$

such that if the number of samples per client is greater than S, then we have with probability at least $1-\delta$ for all ϕ, $\mathbf{v}$, φ,

$$|\sum_i^m\frac{n_i}{N}(\mathcal{L}_i(h(\varphi_i^*;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i))|\leq\epsilon,\tag{35}$$

where ϕ^*, $\mathbf{v}^*$, φ^* are the optimal parameters corresponding to the distribution of each individual client, respectively.

Proof. First, we define the distance between (ϕ, $\mathbf{v}$, φ) and (ϕ^*, $\mathbf{v}^*$, φ^*) as

$$\begin{aligned}&d\big((\phi,\mathbf{v},\varphi),(\phi^*,\mathbf{v}^*,\varphi^*)\big)\\ =&\Big|\sum_i^m\frac{n_i}{N}(\mathcal{L}_i(h(\varphi_i^*;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i))\Big|,\end{aligned}\tag{36}$$

where $N=\sum_{i=1}^m n_i$. By the Theorem 4 from (Baxter 2000), we can find an ϵ-covering in $d((\phi,\mathbf{v},\varphi),(\phi^*,\mathbf{v}^*,\varphi^*))$. Then, according to the notations used in our paper, we have that $S=\mathcal{O}(\frac{1}{m\epsilon^2}log(\frac{\mathcal{C}(\epsilon,\mathcal{H}_l^n)}{\delta}))$, where $\mathcal{C}(\epsilon,\mathcal{H}_l^n)$ is the covering number of $\mathcal{H}_l^n$. In our case, each element of $\mathcal{H}_l^n$ is parameterized by $\phi,\mathbf{v},\varphi$. Therefore, from the triangle inequality and the Lipschitz conditions in Assumption 5, we can get

$$\begin{aligned}&d\big((\phi,\mathbf{v},\varphi),(\phi^*,\mathbf{v}^*,\varphi^*)\big)\\ =&|\sum_i^m\frac{n_i}{N}(\mathcal{L}_i(h(\varphi_i^*;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i))|\\ =&|\sum_i^m\frac{n_i}{N}(\mathcal{L}_i(h(\varphi_i^*;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i^*),\phi_i^*)\\ &+\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i^*)+\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i^*)\\ &-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i))|\\ \leq&\sum_i^m\frac{n_i}{N}|(\mathcal{L}_i(h(\varphi_i^*;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i^*),\phi_i^*)\\ &+\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i^*),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i^*)\\ &+\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i^*)-\mathcal{L}_i(h(\varphi_i;\mathbf{v}_i),\phi_i))|\\ \leq&\sum_{i=1}^m\frac{n_i}{N}(L_h||h(\varphi_i^*;\mathbf{v}_i^*)-h(\varphi_i;\mathbf{v}_i^*)||+L_h||h(\varphi_i;\mathbf{v}_i^*)\\ &-h(\varphi_i;\mathbf{v}_i)||+L_\phi||\phi_i^*-\phi_i||)\\ \leq&L_hL_\varphi||\varphi_i^*-\varphi_i||+L_hL_v||\mathbf{v}_i^*-\mathbf{v}_i||+L_\phi||\phi_i^*-\phi_i||.\end{aligned}\tag{37}$$

Now if there is a parameter space such that ϕ_i, $\mathbf{v}_i$ and φ_i have corresponding optimal point ϕ_i^*, $\mathbf{v}_i^*$ and φ_i^*, which

are $\frac{\epsilon}{L_h L_\varphi + L_h L_v + L_\phi}$ away, respectively, we can get an upper bound of the distance between our model and optimal model, which is an ϵ-covering in $d((\phi, \mathbf{v}, \varphi), (\phi^*, \mathbf{v}^*, \varphi^*))$ matrix. From here we see that $\log(\mathcal{C}(\epsilon, \mathcal{H}_l^n)) = \mathcal{O}(m(d + H + K)\log(\frac{RL_h(L_\varphi + L_v) + RL_\phi}{\epsilon}))$. □

Theorem 2 suggests that S is influenced by several factors: the dimension of the parameters space, the number of clients, and the values of the Lipschitz constants. Specifically, the first part of right hand side of Eq. (34) is determined by the dimensions of the embedding vectors, hypernetwork parameters, and classifier parameters. This component is independent of the number of clients m, as each client has its unique embedding vector, hypernetwork and classifier. Additionally, this theorem points out that generalization depends on the Lipschitz constants, which influence the effective space reachable by the personalized models of clients. This indicates a trade-off between the generalization ability and the flexibility of the personalized model.

C Related Work

Gradient Inversion Attacks. Gradient Inversion Attacks (GIA) (Fredrikson, Jha, and Ristenpart 2015; Zhu, Liu, and Han 2019) is a class of adversarial attacks that exploit the gradients of a machine learning model to infer sensitive information about the training data by leveraging the fact that gradients contain information about the relationship between the input and the model's output. The basic idea behind GIA is to intentionally modify the input data in a way that maximizes the magnitude of the gradients with respect to the sensitive information of interest. By iteratively adjusting the input data based on the gradients, an attacker can gradually approximate the sensitive information, such as private attributes or training data samples, that the model was trained on (Phong et al. 2017; Zhu, Liu, and Han 2019; Geiping et al. 2020; Yin et al. 2021; Luo et al. 2022; Geng et al. 2023; Kariyappa et al. 2023). GIA can pose a significant threat to privacy in scenarios where the model is used in sensitive applications or when the model's training data contains sensitive information. These attacks highlight the need for robust privacy protection mechanisms to mitigate the risk of information leakage through gradients.

Privacy Protection in Federated Learning. Although the local data are not exposed in FL, the exchanged model gradients may still leak sensitive information about the data that can be leveraged by GIA to recover them (Geiping et al. 2020; Huang et al. 2021; Li et al. 2022a; Hatamizadeh et al. 2023; Kariyappa et al. 2023). To further protect the data privacy, additional defense methods have been integrated into FL, and can be categorized into three classes: Secure Multi-party Computing (SMC) (Yao 1982) based methods (Bonawitz et al. 2017; Mugunthan et al. 2019; Mou et al. 2021; Xu et al. 2022), Homomorphic Encryption (HE) (Gentry 2009) based methods (Zhang et al. 2020a,b; Ma et al. 2022; Park and Lim 2022) and Differential Privacy (DP) (Dwork 2006) based methods (Geyer, Klein, and Nabi 2017; McMahan et al. 2018; Yu, Bagdasaryan, and Shmatikov 2020; Bietti et al. 2022; Shen et al. 2023). SMC, originating from Yao's Millionaire problem (Yao 1982), is a framework that aims to protect the input data of each participating party by employing encryption techniques during collaborative computations. With the development of FL, SMC techniques have evolved and been adapted to federated systems to enhance the protection of sensitive data through parameter encryption (Bonawitz et al. 2017; Mugunthan et al. 2019; Mou et al. 2021; Xu et al. 2022). HE, introduced by Gentry (Gentry 2009), is an encryption algorithm that preserves the homomorphic property of ciphertexts. In the context of FL, HE enables the central server to perform algebraic operations directly on encrypted parameters without the need for decryption (Zhang et al. 2020a,b; Ma et al. 2022; Park and Lim 2022). DP is a widely adopted privacy-preserving technique in both industry and academia by clipping the gradients and adding noise to personal sensitive attribute (Dwork 2006; Abadi et al. 2016). In the context of FL, DP is employed to prevent inverse data retrieval by clipping gradients and adding noise to participants' uploaded parameters (Geyer, Klein, and Nabi 2017; McMahan et al. 2018; Yu, Bagdasaryan, and Shmatikov 2020; Bietti et al. 2022; Shen et al. 2023). However, SMC and HE methods are unsuitable to DNN models due to their extremely high computation and communication cost, while DP methods usually introduce additional computation cost and result in a decrease in model performance (Bonawitz et al. 2017; Zhang et al. 2020b; Geyer, Klein, and Nabi 2017; McMahan et al. 2018). Apart from these defense methods with theoretical guarantees, there are other empirical yet effective defense strategies, such as gradient pruning/masking (Zhu, Liu, and Han 2019; Huang et al. 2021; Li et al. 2022b), noise addition (Zhu, Liu, and Han 2019; Wei et al. 2020; Huang et al. 2021; Li et al. 2022b), Soteria (Sun et al. 2020), PRECODE (Scheliga, Mäder, and Seeland 2022), and FedKL (Ren et al. 2023). However, these methods still suffer from privacy-utility trade-off problems, as shown in Tables 2 and 5 in (Huang et al. 2021) for gradient pruning/masking, Tables 1, 2, and 3 in PRECODE (Scheliga, Mäder, and Seeland 2022), Table 1 in FedKL (Ren et al. 2023), and Figure 5 in Soteria (Sun et al. 2020). In contrast to these approaches, with the help of hypernetworks (Ha, Dai, and Le 2017), this work proposes a novel FL framework that effectively "breaks the direct connection" between the shared parameters and the local private data to defend against GIA while achieving a favorable privacy-utility trade-off.

Hypernetworks in Federated Learning. Hypernetworks (Ha, Dai, and Le 2017) are deep neural networks that generate the weights for another network, known as the target network, based on varying inputs to the hypernetwork. Recently, there have been some works that incorporate hypernetworks into FL for learning personalized models (Shamsian et al. 2021; Carey, Du, and Wu 2022; Li et al. 2023b; Tashakori et al. 2023; Lin et al. 2023). All of these methods adopt the similar idea that a central hypernetwork model are trained on the server to generate a set of models, one model for each client, which aims to generate personalized model for each client. Since the hypernetwork and client em-

beddings are trained on the server, which makes the server possessing all the information about the local models, enabling the server to recover the original inputs by GIA ((see Table 3 and Figure 5 in Appendix)). In contrast to existing approaches, this work presents a Hypernetwork Federated Learning (HyperFL) framework, which prioritizes data privacy preservation over personalized model generation through the utilization of hypernetworks.

D Additional Experimental Results and Experimental Details.

D.1 Details of Experimental Setup

Datasets. For the Main Configuration HyperFL, we evaluate our method on four widely-used image classification datasets: (1) EMNIST (Extended MNIST) (Cohen et al. 2017), a dataset with 62 categories of handwritten characters, including 10 digits, 26 uppercase letters, and 26 lowercase letters; (2) Fashion-MNIST (Xiao, Rasul, and Vollgraf 2017), a dataset designed for fashion product images, containing 10 categories of clothing items; (3) CIFAR-10 (Krizhevsky, Hinton et al. 2009), a widely used benchmark dataset for image classification tasks, consisting of 60,000 color images distributed across 10 different classes; and (4) CINIC-10 (Darlow et al. 2018), a composite image dataset that combines samples from CIFAR-10 and ImageNet (Russakovsky et al. 2015), comprising 270,000 images spanning 10 different classes.

Similar to (Karimireddy et al. 2020; Zhang et al. 2021; Xu, Tong, and Huang 2023), we create a non-IID data distribution by ensuring all clients have the same data size, in which $s\%$ of data (20% by default) are uniformly sampled from all classes and the remaining $(100 - s)\%$ from a set of dominant classes for each client. Following (Xu, Tong, and Huang 2023), we evenly divide all clients into multiple groups, with each group having the same dominant classes. Specifically, for the 10-category Fashion-MNIST, CIFAR-10 and CINIC-10 datasets, we divide clients into 5 groups. Each group is assigned three consecutive classes as the dominant class set, starting from class 0, 2, 4, 6, and 8 for the respective groups. For EMNIST dataset, we divide clients into 3 groups, with each group assigned the dominant set of digits, uppercase letters, and lowercase letters, respectively.

For the HyperFL-LPM, we evaluate our method on the EMNIST (Cohen et al. 2017) and CIFAR-10 (Krizhevsky, Hinton et al. 2009) datasets.

Model Architectures. For the Main Configuration HyperFL, simlar to (Xu, Tong, and Huang 2023), we adopt two different CNN target models for EMNIST/Fashion-MNIST and CIFAR-10/CINIC-10, respectively. The first CNN target model is built with two convolutional layers. The first CNN target model is built with two convolutional layers (16 and 32 channels) followed by max pooling layers, two fully-connected layers (128 and 10 units), and a softmax output layer, using LeakyReLU activation functions (Xu et al. 2015). The second CNN model is similar to the first one but adds one more 64-channel convolution layer. The hypernetwork is a fully-connected neural network with one hidden layer, multiple linear heads per target weight tensor. The client embeddings are learnable vectors with dimension equals 64.

For the HyperFL-LPM, we adopt the ViT-S/16 (Dosovitskiy et al. 2021) and ResNet-18 (He et al. 2016) pre-trained on the ImageNet dataset (Deng et al. 2009) as the feature extractor. When the pre-trainde model is ResNet, the adapter is inserted behind each resnet block. The adapter within each transformer block consists of a down-projection layer, ReLU activation functions (Nair and Hinton 2010), and a up-projection layer. The hypernetwork is a fully-connected neural network with one hidden layer, multiple linear heads per target weight tensor. The client embeddings are learnable vectors with dimension equals 64.

Compared Methods. For the Main Configuration HyperFL, we compare the proposed method with the following approaches: (1) Local-only, where clients train models locally without collaboration; (2) FedAvg (McMahan et al. 2017), a widely-used FL method; (3) pFedHN (Shamsian et al. 2021), that utilizes a central hypernetwork model trained on the server to generate a set of models, one model for each client; and some DP-based FL methods, including (4) DP-FedAvg (McMahan et al. 2018), which incorporates differential privacy into FedAvg; (5) PPSGD (Bietti et al. 2022), a personalized private SGD algorithm with user-level differential privacy; and (6) CENTAUR (Shen et al. 2023), which trains a single differentially private global representation extractor while allowing personalized classifier heads. However, all these compared DP-based FL methods (i.e., DP-FedAvg, PPSGD, and CENTAUR) focus on user-level DP setting (McMahan et al. 2018), which cannot guarantee protection against *honest-but-curious* server attacks as they upload original gradients to the server. Therefore, we adapt these methods to fit the ISRL-DP setting (Lowy and Razaviyayn 2023), where users trust their own client but not the server or other clients, thereby defending against *honest-but-curious* server attacks.

For the HyperFL-LPM, we compare our method with (1) Local-only with fixed feature extractor; (2) Local-only with adapter fine-tuning; (3) FedAvg with fixed feature extractor; and (4) FedAvg with adapter fine-tuning.

Training Settings. For the Main Configuration HyperFL, mini-batch SGD (Ruder 2016) is adopted as the local optimizer for all approaches. Similar to (Xu, Tong, and Huang 2023), we set the step sizes η_h and η_v for local training to 0.01 for EMNIST/Fashion-MNIST and 0.02 for CIFAR-10/CINIC-10. The setp size η_g is set to 0.1 for all the datasets. The weight decay is set to 5e-4 and the momentum is set to 0.5. The batch size is fixed to B = 50 for all datasets except EMNIST (B = 100). The client embedding dimension is set to 64. The number of local training epochs is set to 5 for all FL approaches and the number of global communication rounds is set to 200 for all datasets. Furthermore, following (Xu, Tong, and Huang 2023), we conduct experiments on two setups, where the number of clients is 20 and 100, respectively. For the latter, we apply random client selection with sampling rate 0.3 along with full participation in the last round. The training data size per client

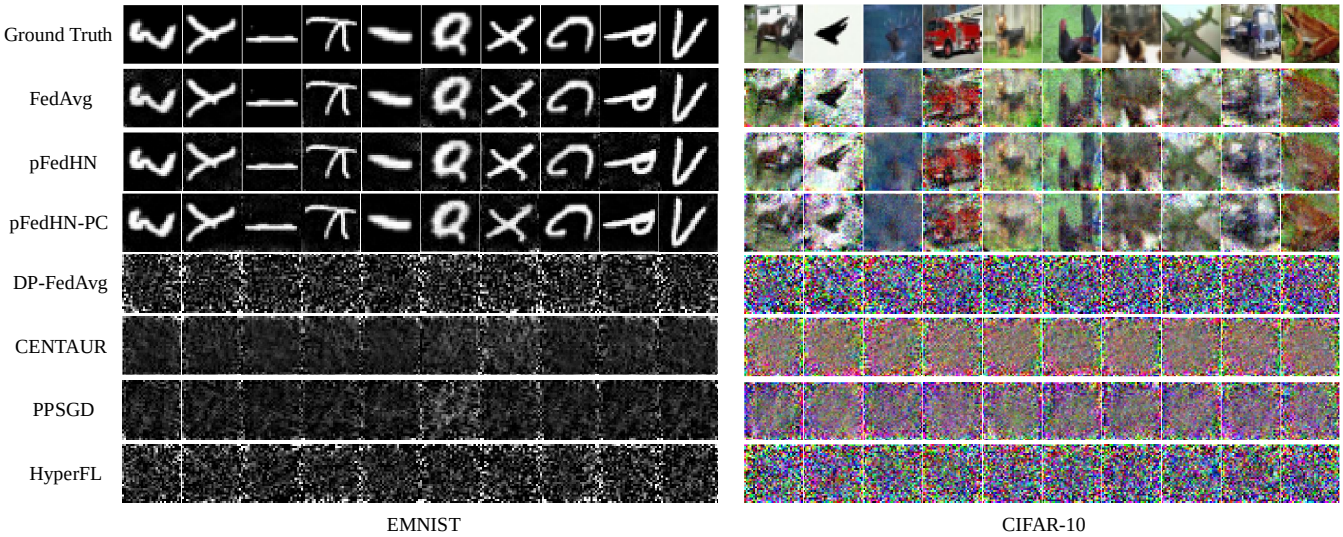


Figure 5: Reconstructed images of IG.

is set to 600 for all datasets except EMNIST, where the size is 1000. For the DP-based FL methods, the DP budget ϵ is set to 4 and the Gaussian noise σ is $1e-5$ to satisfy the (ϵ, σ) privacy guarantee. Average test accuracy of all local models is reported for performance evaluation.

For the HyperFL-LPM, we conducted experiments with 20 clients. Furthermore, differently from HyperFL, batch size 16 is adopted for all datasets. The step sizes η_h and η_v for local training are 0.02 for EMNIST and 0.1 for CIFAR-10 when using ViT pre-trained models. When using ResNet pre-trained models, the step size is set to 0.01 for all datasets.

Privacy Evaluation. EMNIST and CIFAR-10 are used to evaluate privacy preservation capability of the proposed HyperFL. We choose a subset of 50 images from each dataset to evaluate the privacy leakage. A batch size of one is used. For experimental comparison, we set all the unknown variable in HyperFL are learnable and optimized simultaneously for IG (Geiping et al. 2020) and ROG (Yue et al. 2023). For the optimization of IG (Geiping et al. 2020), we optimize the attack for 10,000 iterations using the Adam optimizer (Kingma and Ba 2015), with an initial learning rate of 0.1. The learning rate is decayed by a factor of 0.1 at 3/8, 5/8, and 7/8 of the optimization process. The coefficient of the TV regularization term is set to 1e-6. For the optimization of ROG (Yue et al. 2023), the Adam optimizer (Kingma and Ba 2015) with a learning rate of 0.05 is adopted, and the total number of iterations is set to 100. To further demonstrate the privacy preservation capability of the proposed HyperFL, we design a tailored attack method. Specifically, we first recover the client embedding according to Eq. (10), and then recover the input data by solving the upper-level subproblem in objective Eq. (9). Since Δ_θ cannot be obtained [2], we utilize model inversion attack methods (He, Zhang, and Lee 2019, 2020; Jiang, Zhou, and Grossklags 2022; Erdoğan, Küpçü, and Çiçek 2022) to solve the upper-level subproblem in objective Eq. (9). Peak signal to noise ratio (PSNR) (Hore and Ziou 2010), structural similarity (SSIM) (Wang et al. 2004), and learned perceptual image patch similarity (LPIPS) (Zhang et al. 2018) are adopted as the metrics for reconstruction attacks on image data. Lower LPIPS, higher PSNR and SSIM of reconstructed images indicate better attack performance.

All experiments are conducted on NVIDIA GeForce RTX 3090 GPUs.

[2]The client embedding changes at each iteration, making Δ_θ impossible to calculate.

D.2 Additional Experimental Results

Privacy Evaluation. The visualization results of IG (Geiping et al. 2020) of the first 10 images are provided in Figure 5. From this figure, we can observe that the native FedAvg and pFedHN methods have a much higher risk of leaking data information, as indicated by the reconstructed images closely resembling the original ones. Although introducing DP improves data privacy, there is a significant drop in model performance, as shown in Table 1. In contrast, HyperFL achieves a similar level of privacy protection while outperforming all DP-based methods and the native FedAvg in terms of model accuracy.

The reconstructed and visualization results of ROG (Yue et al. 2023) are provided in Table 5 and Figure 6. From these results, we can observe that the proposed HyperFL can also defend against SOTA attack method.

	EMNIST			CIFAR-10		
Method	PSNR	SSIM	LPIPS	PSNR	SSIM	LPIPS
FedAvg	24.26	0.9516	0.3024	23.09	0.9228	0.4363
HyperFL	3.44	0.0459	0.7883	7.78	0.0137	0.7802

Table 5: Reconstruction results of ROG.

The reconstructed visualization results of the tailored attack method are presented in Figure 7. These results demonstrate that even the tailored attack method is unable to re-

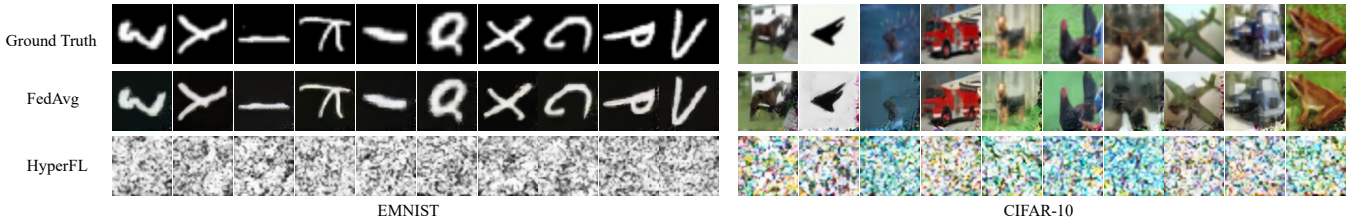


Figure 6: Reconstructed images of ROG.

cover any information from the proposed HyperFL framework, thereby showcasing the robust privacy preservation capability of HyperFL.

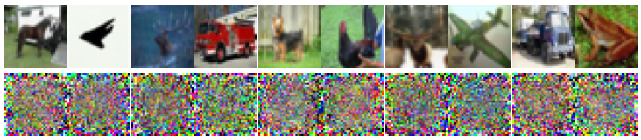
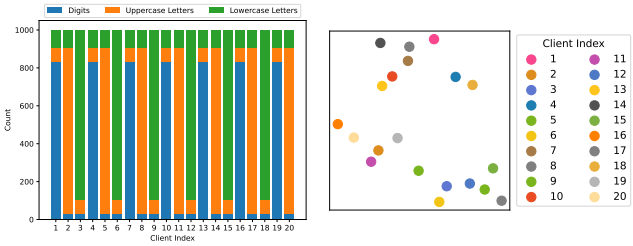


Figure 7: Reconstructed images of the tailored attack method. The first row contains the original images, while the second row shows the reconstruction results.

Learned Client Embeddings. In our experiments, we learn to represent each client using a trainable embedding vector $\mathbf{v}_i$. These embedding vectors are randomly initialized to the same value. By setting these embedding vectors trainable, they can learn a continuous semantic representation over the set of clients. The t-SNE visualization (Van der Maaten and Hinton 2008) of the learned client embeddings of the EMNIST dataset with 20 clients is shown in Figure 8(b). Form this figure we can see that there is a distinct grouping of the learned client embeddings into three clusters, which aligns with the data partitioning we employed, as shown in Figure 8(a). This phenomenon demonstrates the meaningfulness of the learned client embeddings in capturing the underlying relationship of the clients. In this way, personalized feature extractor parameters for each client can be generated by taking the meaningful client embedding as input for the hypernetwork. Therefore, the model can achieve better performance by adopting personalized feature extractors.



(a) (b)

Figure 8: (a) Label distribution of the EMNIST dataset with 20 clients. (b) The t-SNE visualization of the learned client embeddings of the EMNIST dataset with 20 clients.